```
import { AbilityConstant, UIAbility, Want, common, wantConstant } from '@kit.AbilityKit'
import { window } from '@kit.ArkUI'
import { BusinessError } from '@kit.BasicServicesKit'
import { CommonConstants } from '../common/constants/CommonConstants'
import { StorageManager } from '../common/utils/StorageManager'
import { settingsVM } from '../viewmodel/SettingsViewModel'
import { homeVM } from '../viewmodel/HomeViewModel'
import { routeVM } from '../viewmodel/RoutePageViewModel'
import { Logger } from '../common/utils/Logger'
import { safeAreaHelper } from '../common/utils/SafeAreaHelper'
import { deadlineReminderService } from '../common/services/DeadlineReminderService'
import { pomodoroService } from '../common/services/PomodoroService'
import { notificationCenterService } from '../common/services/NotificationCenterService'
import { DeepSeekService } from '../common/services/DeepSeekService'
import { globalNavStack, setMigrationPayload, getCurrentTabIndex, navigateToWithIdOnce, markPendingDeepLink
} from '../common/RouterManager'
import { util } from '@kit.ArkTS';
import { CloudSdkService } from '../common/services/CloudSdkService'
import { FeedbackService } from '../common/services/FeedbackService'
/** 通知直达路由信息（冷启动暂存，待 UI 加载后执行） */
interface PendingNotificationRoute {
  routeName: string
  id: string
  /** true 表示落地即呼出全屏番茄钟（番茄钟倒计时通知专用） */
  openImmersive: boolean
}
/** 多端协同迁移负载 */
interface MigrationPayload {
  routeName: string
  param: string
  tabIndex: number
}
export default class EntryAbility extends UIAbility {
  private readonly tag: string = 'EntryAbility'
  /** 待执行的通知直达路由 */
  private pendingNotificationRoute: PendingNotificationRoute | null = null
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    Logger.info(this.tag, 'onCreate, launchReason=' + launchParam.launchReason)
    // 隐私合规：want.parameters 在多端流转场景携带完整用户数据（migrationData），
    // 禁止走 Logger.info（%{public}s 公开可读）。debug 档为 %{private}s，release 掩码。
    Logger.debug(this.tag, 'want.parameters=' + JSON.stringify(want.parameters))
    // 优先级：通知直达 > 多端流转
    if (!this.extractNotificationRoute(want)) {
      this.extractMigrationData(want)
    }
    // 初始化所有服务，确保应用启动时就准备就绪
    this.initializeAppServices();
  }
  /**
   * 本地服务初始化（隐私合规拆分后仅保留本地链路）
```

```
   * 云端 SDK（AGC auth / CloudFoundationKit / CloudDB）不在冷启动初始化——
   * 必须等用户在 MainTab 同意《隐私协议》后，由 CloudSdkService 统一拉起。
   */
  private async initializeAppServices(): Promise<void> {
    try {
      // 1. 初始化本地存储
      await StorageManager.init(this.context);
      // 2. 初始化其他服务（如截止日期提醒）
      deadlineReminderService.init(this.context);
      // 3. 初始化番茄钟服务（前台计时 + 后台 reminderAgent + 通知）
      pomodoroService.init(this.context);
      // 3.1 初始化站内通知中心（展示/删除/直达）
      notificationCenterService.init(this.context);
      // 4. 初始化 DeepSeek 服务（注入 context 用于 i18n 资源读取；实际云端调用依赖隐私同意后的
CloudSdkService）
      DeepSeekService.getInstance().init(this.context);
      Logger.info(this.tag, 'Local application services initialized successfully');
    } catch (e) {
      Logger.error(this.tag, 'Error initializing application services: ' + (e as Error).message);
    }
  }
  onContinue(wantParam: Record<string, Object>): AbilityConstant.OnContinueResult {
    Logger.info(this.tag, 'onContinue')
    wantParam[wantConstant.Params.SUPPORT_CONTINUE_PAGE_STACK_KEY] = false
    let routeName: string = 'MainTab'
    let paramStr: string = '{}'
    let tabIndex: number = 0
    try {
      const pathStack: NavPathInfo[] = globalNavStack.getPathStack()
      if (Array.isArray(pathStack) && pathStack.length > 0) {
        const topPath: NavPathInfo = pathStack[pathStack.length - 1]
        if (topPath && typeof topPath.name === 'string' && topPath.name.length > 0) {
          routeName = topPath.name
        }
        if (topPath.param !== undefined && topPath.param !== null) {
          try {
            paramStr = JSON.stringify(topPath.param) ?? '{}'
          } catch (_e) {
            paramStr = '{}'
          }
        }
      } else {
        try {
          tabIndex = getCurrentTabIndex()
        } catch (_e) {
          tabIndex = 0
        }
      }
    } catch (_e) {
      Logger.warn(this.tag, 'onContinue: getPathStack failed')
```

```
    }
    const payload: MigrationPayload = {
      routeName: routeName,
      param: paramStr,
      tabIndex: tabIndex
    }
    const migrationData: string = JSON.stringify(payload)
    Logger.info(this.tag, 'onContinue: migrationData=' + migrationData)
    wantParam.routeName = routeName
    wantParam.param = paramStr
    wantParam.tabIndex = tabIndex
    const encoder: util.TextEncoder = new util.TextEncoder()
    const encoded: Uint8Array = encoder.encodeInto(migrationData)
    wantParam.migrationData = encoded.buffer as ArrayBuffer
    Logger.info(this.tag, 'onContinue: routeName=' + routeName + ', tabIndex=' + tabIndex)
    return AbilityConstant.OnContinueResult.AGREE
  }
  onNewWant(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    Logger.info(this.tag, 'onNewWant, launchReason=' + launchParam.launchReason)
    // 隐私合规：同 onCreate —— parameters 可能含完整流转数据，降 debug（%{private}s）
    Logger.debug(this.tag, 'want.parameters=' + JSON.stringify(want.parameters))
    // 优先级：通知直达 > 多端流转
    // 热启动时 UI 已加载，解析后立即跳转
    if (!this.extractNotificationRoute(want)) {
      this.extractMigrationData(want)
    } else {
      this.executeNotificationRoute()
    }
    // 热启动并集合并刷新（方案A）：不无脑用云端覆盖本地，两端独有数据都保留
    // 冷启动已由 settingsVM.init() → smartSync('merge') 自动执行
    settingsVM.triggerSync()
    // 番茄钟运行中检测：热启动时若专注进行中，自动呼出沉浸屏
    this.restorePomodoroImmersiveIfNeeded()
  }
  /**
   * 番茄钟沉浸屏恢复
   * 热启动时若番茄钟运行中，导航到对应任务详情页（页面内番茄钟卡片会显示进行中状态）
   *
   *      这里【不】自动 showImmersive：用户自己回前台被打断体验差，
   *    且全局可见标志残留曾导致"打开任意任务详情都被番茄钟覆盖"。
   *    需要"落地即全屏"的只有一种情况——用户主动点番茄钟通知，
   *    那条路径由 executeNotificationRoute 依据 openImmersive 标记显式呼出。
   */
  private restorePomodoroImmersiveIfNeeded(): void {
    try {
      if (!pomodoroService.isRunning()) {
        return
      }
      const session = pomodoroService.getCurrentSession()
      if (session === null) {
```

```
      return
    }
    Logger.info(this.tag, 'restorePomodoroImmersiveIfNeeded: pomodoro running, taskId=' + session.taskId)
    // 检查页面栈顶是否已是 TaskDetailPage，避免重复压栈
    // 注意这里【只比路由名、不比 id】是刻意的：栈顶已是"某个任务的详情页"时就不打扰用户，
    // 若改成 name+id 比较，会在"番茄钟跑 B、用户正在看 A 的详情"时把页面替用户换掉。
    const pathStack: NavPathInfo[] = globalNavStack.getPathStack()
    if (Array.isArray(pathStack) && pathStack.length > 0) {
      const topPath: NavPathInfo = pathStack[pathStack.length - 1]
      if (topPath && typeof topPath.name === 'string' && topPath.name === 'TaskDetailPage') {
        Logger.info(this.tag, 'restorePomodoroImmersiveIfNeeded: already on TaskDetailPage, skip')
        return
      }
    }
    // 导航到 TaskDetailPage（携带 taskId，页面从 homeVM 反查构造数据）；幂等跳转避免重复压栈
    navigateToWithIdOnce('TaskDetailPage', session.taskId)
    Logger.info(this.tag, 'restorePomodoroImmersiveIfNeeded: navigate to TaskDetailPage')
  } catch (e) {
    Logger.warn(this.tag, 'restorePomodoroImmersiveIfNeeded failed: ' + (e as Error).message)
  }
}
private extractMigrationData(want: Want): void {
  if (!want.parameters) {
    Logger.info(this.tag, 'extractMigrationData: no parameters')
    return
  }
  try {
    const params: Record<string, Object> = want.parameters
    Logger.info(this.tag, 'extractMigrationData: keys=' + Object.keys(params).join(','))
    const migrationDataRaw: Object | undefined = params.migrationData
    if (migrationDataRaw !== undefined && migrationDataRaw !== null) {
      try {
        let jsonStr: string = ''
        if (migrationDataRaw instanceof ArrayBuffer) {
          const bytes: Uint8Array = new Uint8Array(migrationDataRaw)
          const decoder: util.TextDecoder = util.TextDecoder.create('utf-8')
          jsonStr = decoder.decodeToString(bytes)
        } else if (typeof migrationDataRaw === 'string') {
          jsonStr = migrationDataRaw
        }
        if (jsonStr.length > 0) {
          const parsed: Record<string, Object> = JSON.parse(jsonStr) as Record<string, Object>
          const routeName: string = (parsed.routeName as string) ?? 'MainTab'
          const paramStr: string = (parsed.param as string) ?? '{}'
          const tabIndex: number = (parsed.tabIndex as number) ?? 0
          setMigrationPayload({ routeName, param: paramStr, tabIndex })
          markPendingDeepLink()
          Logger.info(this.tag, 'extractMigrationData from migrationData SUCCESS: routeName=' + routeName)
          return
        }
```

```
      } catch (e) {
        Logger.warn(this.tag, 'extractMigrationData: migrationData parse failed: ' + (e as Error).message)
      }
    }
    const routeName: string = this.decodeParamValue(params.routeName)
    if (routeName.length === 0) {
      Logger.info(this.tag, 'extractMigrationData: routeName is empty, skip')
      return
    }
    const paramStr: string = this.decodeParamValue(params.param)
    let tabIndex: number = 0
    const tabIndexVal: Object | undefined = params.tabIndex
    if (typeof tabIndexVal === 'number') {
      tabIndex = tabIndexVal
    } else if (typeof tabIndexVal === 'string') {
      tabIndex = parseInt(tabIndexVal, 10) || 0
    }
    setMigrationPayload({ routeName, param: paramStr, tabIndex })
    markPendingDeepLink()
    Logger.info(this.tag, 'extractMigrationData from keys SUCCESS: routeName=' + routeName)
  } catch (_e) {
    Logger.warn(this.tag, 'extractMigrationData failed')
  }
}
private decodeParamValue(value: Object | undefined): string {
  if (value === undefined || value === null) {
    return ''
  }
  if (typeof value === 'string') {
    return value
  }
  if (value instanceof ArrayBuffer) {
    try {
      const bytes: Uint8Array = new Uint8Array(value)
      let result: string = ''
      for (let i: number = 0; i < bytes.length; i++) {
        result += String.fromCharCode(bytes[i])
      }
      return result
    } catch (_e) {
      return ''
    }
  }
  if (typeof value === 'object') {
    try {
      return JSON.stringify(value)
    } catch (_e) {
      return ''
    }
  }
```

```
    return ''
  }
  /**
   * 解析通知直达路由
   * 通知 Want 携带 parameters: { routeName: 'GoalDetailPage'|'TaskDetailPage', goalId|taskId: xxx }
   * 区别于多端协同迁移（携带 migrationData / param / tabIndex）
   *
   * @param want - 启动 Want
   * @returns true 表示为通知直达场景，已暂存路由；false 表示非通知场景
   */
  private extractNotificationRoute(want: Want): boolean {
    if (!want.parameters) {
      return false
    }
    try {
      const params: Record<string, Object> = want.parameters
      const routeName: string = this.decodeParamValue(params.routeName)
      if (routeName === 'GoalDetailPage') {
        const goalId: string = this.decodeParamValue(params.goalId)
        if (goalId.length > 0) {
          this.pendingNotificationRoute = { routeName: routeName, id: goalId, openImmersive: false }
          markPendingDeepLink()
          Logger.info(this.tag, 'extractNotificationRoute: goal detail, goalId=' + goalId)
          return true
        }
      }
      if (routeName === 'TaskDetailPage') {
        const taskId: string = this.decodeParamValue(params.taskId)
        if (taskId.length > 0) {
          const openImmersive: boolean = this.decodeParamValue(params.openImmersive) === '1'
          this.pendingNotificationRoute = {
            routeName: routeName, id: taskId, openImmersive: openImmersive
          }
          markPendingDeepLink()
          Logger.info(this.tag, 'extractNotificationRoute: task detail, taskId=' + taskId +
            ', openImmersive=' + openImmersive.toString())
          return true
        }
      }
      return false
    } catch (_e) {
      Logger.warn(this.tag, 'extractNotificationRoute failed: ' + (_e as Error).message)
      return false
    }
  }
  /**
   * 执行暂存的通知直达路由跳转
   * 冷启动：在 loadContent 回调中延迟调用（等待 MainTab 渲染）
   * 热启动：在 onNewWant 中直接调用
   */
```

```
  private executeNotificationRoute(): void {
    if (this.pendingNotificationRoute === null) {
      return
    }
    const route: PendingNotificationRoute = this.pendingNotificationRoute
    this.pendingNotificationRoute = null
    try {
      // 幂等跳转：栈顶已是同一 (route, id) 时不再压栈。
      // 否则每点一次通知压一层新详情页，用户"返回一次"会退回到上一个同类页面而非来源页。
      const navigated: boolean = navigateToWithIdOnce(route.routeName, route.id)
      Logger.info(this.tag, 'executeNotificationRoute: navigate to ' + route.routeName +
        ', id=' + route.id + ', effective=' + navigated.toString())
      // 番茄钟通知直达：确认目标页已在位后再置位"全屏可见"，
      // 新页面 onDestinationReady 会据此直接渲染全屏番茄钟，不再依赖进程内标志是否残留
      // （进程被回收后该标志为 false，点通知只会落到任务详情页）。
      // 顺序要求：必须在导航生效【之后】——若跳转被 600ms 防抖拦下，
      // 提前置位会让标志悬挂（没有宿主页面承载全屏层），之后用户手动打开该任务详情会被"意外全屏"。
      if (navigated && route.openImmersive && route.routeName === 'TaskDetailPage') {
        const session = pomodoroService.getCurrentSession()
        if (session !== null && session.state !== 'idle' && session.taskId === route.id) {
          pomodoroService.showImmersive()
          Logger.info(this.tag, 'executeNotificationRoute: session active, show immersive screen')
        }
      }
    } catch (e) {
      Logger.error(this.tag, 'executeNotificationRoute failed: ' + (e as Error).message)
    }
  }
  // 加载页面内容
  private setupWindowStage(windowStage: window.WindowStage): void {
    windowStage.loadContent('pages/MainTab', (err: BusinessError): void => {
      if (err.code) {
        Logger.error(this.tag, 'loadContent FAILED, code=' + err.code + ', msg=' + err.message)
        return
      }
      Logger.info(this.tag, 'loadContent SUCCESS')
      // 冷启动通知直达：待 MainTab 渲染后执行暂存的通知路由
      if (this.pendingNotificationRoute !== null) {
        setTimeout((): void => {
          this.executeNotificationRoute()
        }, CommonConstants.COLD_START_ROUTE_DELAY_MS)
      }
    })
  }
  /**
   * 初始化窗口安全区与沉浸式布局
   * - 开启窗口全屏布局，使应用内容可延伸到状态栏/底部手势区
   * - 存储 statusBarHeight / bottomSafeAreaHeight 到 AppStorage，供全页面统一避让
   */
  private initWindowInsets(windowStage: window.WindowStage): void {
```

```
    const mainWindow = windowStage.getMainWindowSync()
    // 关键：开启全屏布局，否则 expandSafeArea 无法突破系统默认 inset
    mainWindow.setWindowLayoutFullScreen(true)
    // 自愈：每次启动都无条件恢复"状态栏 + 导航栏"可见。
    // 番茄钟沉浸屏会在整屏生命周期内隐藏系统栏；若进程在隐藏态被杀/崩溃，
    // 这里保证下次启动一定回到可见状态，不会留下"应用没有状态栏"这类诡异故障。
    const barNames: Array<'status' | 'navigation'> = ['status', 'navigation']
    mainWindow.setWindowSystemBarEnable(barNames).catch((e: BusinessError): void => {
      Logger.warn(this.tag, 'restore system bars failed: ' + e.message)
    })
    const systemAvoidArea = mainWindow.getWindowAvoidArea(window.AvoidAreaType.TYPE_SYSTEM)
    const statusBarHeight = px2vp(systemAvoidArea.topRect.height)
    const navAvoidArea = mainWindow.getWindowAvoidArea(window.AvoidAreaType.TYPE_NAVIGATION_INDICATOR)
    const bottomSafeAreaHeight = px2vp(navAvoidArea.bottomRect.height)
    // 写入响应式安全区单例，全页面 build 内读取自动刷新
    safeAreaHelper.refresh(statusBarHeight, bottomSafeAreaHeight)
    // 监听避让区变化：旋转/折叠/悬浮态后 statusBarHeight/bottomSafeAreaHeight 失效，
    // 不监听会导致页面顶部/底部避让错位
    mainWindow.on('avoidAreaChange', (data: window.AvoidAreaOptions): void => {
      if (data.type === window.AvoidAreaType.TYPE_SYSTEM) {
        const top: number = px2vp(data.area.topRect.height)
        safeAreaHelper.refresh(top, safeAreaHelper.getBottom())
        Logger.info(this.tag, 'avoidAreaChange: statusBarHeight=' + top.toString())
      } else if (data.type === window.AvoidAreaType.TYPE_NAVIGATION_INDICATOR) {
        const bottom: number = px2vp(data.area.bottomRect.height)
        safeAreaHelper.refresh(safeAreaHelper.getStatusBar(), bottom)
        Logger.info(this.tag, 'avoidAreaChange: bottomSafeAreaHeight=' + bottom.toString())
      }
    })
    Logger.info(this.tag, 'statusBarHeight=' + statusBarHeight.toString() +
      ', bottomSafeAreaHeight=' + bottomSafeAreaHeight.toString())
  }
  onWindowStageCreate(windowStage: window.WindowStage): void {
    Logger.info(this.tag, 'onWindowStageCreate')
    this.initWindowInsets(windowStage)
    this.setupWindowStage(windowStage)
  }
  // 接续场景（多端流转恢复）系统调用此方法，非 onWindowStageCreate
  onWindowStageRestore(windowStage: window.WindowStage): void {
    Logger.info(this.tag, 'onWindowStageRestore')
    this.initWindowInsets(windowStage)
    this.setupWindowStage(windowStage)
  }
  onWindowStageDestroy(): void {
    Logger.info(this.tag, 'onWindowStageDestroy')
  }
  onDestroy(): void {
    Logger.info(this.tag, 'onDestroy')
    settingsVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'settingsVM.flush failed: ' + e.message)
```

```
    })
    homeVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'homeVM.flush failed: ' + e.message)
    })
    routeVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'routeVM.flush failed: ' + e.message)
    })
    // SettingsViewModel 防护：清定时器/退订事件总线，阻断 destroy 后的云同步触发
    settingsVM.destroy()
    CloudSdkService.getInstance().destroy()
  }
  onForeground(): void {
    Logger.info(this.tag, 'onForeground')
    // 回前台：不再取消后台兜底提醒，而是【重新注册】（幂等，内部会先取消旧的）。
    // 原因：取消会留下"前台运行期间进程被系统回收 / 崩溃"的提醒空洞——那条路径不会触发
    // onBackground，兜底就此丢失。重复通知的风险由两处共同消除：
    //   ① 兜底提醒的触发时间带 5s 延迟（PomodoroService.BG_REMINDER_LEAD_MILLIS）；
    //   ② 前台计时器到点时 onPhaseComplete() 会先 cancelReminder() 再发完成通知。
    pomodoroService.registerBackgroundReminder()
    // 回前台：补传上次因未登录/网络异常而暂存的用户反馈（内部有云 SDK 就绪判断，不满足则静默跳过）
    FeedbackService.getInstance().flushQueue().catch((e: Error): void => {
      Logger.warn(this.tag, 'flush feedback queue failed: ' + e.message)
    })
  }
  onBackground(): void {
    Logger.info(this.tag, 'onBackground')
    settingsVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'onBackground settingsVM.flush failed: ' + e.message)
    })
    homeVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'onBackground homeVM.flush failed: ' + e.message)
    })
    routeVM.flush().catch((e: Error): void => {
      Logger.error(this.tag, 'onBackground routeVM.flush failed: ' + e.message)
    })
    // 退后台：注册番茄钟 reminderAgent 兜底提醒
    // 即使进程被杀，系统也会在倒计时结束时弹出通知（应用常驻通知无法保证后台存活）
    pomodoroService.registerBackgroundReminder()
  }
}
/**
 * @file AI 智能拆解页 (Phase 2 - V2 纯净版 + 套娃结构 + 同文件组件化)
 * @description 像素级实现：
 *   - 顶部输入区：标题 + TextArea 多行输入 (maxLength 200) + 字数统计
 *   - 场景芯片区：3-4 个预设目标，点击填入输入框
 *   - 操作按钮区：主按钮"开始 AI 拆解"，Loading 态显示 LoadingProgress
 *   - 结果展示区：统计 + List 卡片（可展开 steps）+ 底部悬浮"一键加入路线"
 *
 * 套娃结构（4 级架构）：
 *   - AI 拆解后生成 1 个 L1 目标（含 3 个 L2 任务，每个 L2 任务含 2-3 个 L3 步骤）
```

```
 *   - 一键加入路线：仅入库 L1 目标到 routeVM（**不再自动创建 L4 待办**）
 *   - L4 待办由用户在目标详情页通过"加入待办"按钮触发摘取流
 *
 * V2 规范：
 *   - @ComponentV2 + @Local
 *   - 严禁 @State / @Prop / @Link
 *   - async/await + try/catch 处理异步
 *   - 严禁 Stack 配 justifyContent
 *   - 严禁将 Promise 赋值给 void
 *
 * 同文件组件化（方案A）：
 *   - 9 个 @ComponentV2 子组件 + 1 个根页面
 *   - 所有 @Local 状态保留在根页面，子组件仅通过 @Param 接收只读数据
 *   - 所有数据操作/状态修改/路由/弹窗通过 @Event 向上回调给根页面
 *   - 列表条目组件使用 @ReusableV2 复用优化
 *
 * 功能保底：
 *   - 多行输入：使用 TextArea（原生支持多行 + maxLength + placeholder）
 *   - 密码框：使用 InputType.Password
 *   - 回车提交：onSubmit((keyCode: EnterKeyType) => {...})
 *
 * @module pages
 * @date 2026-06-19
 */
import { promptAction } from '@kit.ArkUI'
import {
  AiGeneratedTask,
  AIParseResult,
  buildHomeTodoItem,
  ChatMessage,
  ChatRole,
  DeepSeekMessage,
  DeepSeekPlanJson,
  FeedbackRouteNavParam,
  GoalCardItem,
  HomeTodoItem,
  TaskPriority,
  TaskStatus
} from '../model/Models'
import { createGoalFromAi } from '../common/utils/AiGoalFactory'
import { DeepSeekService } from '../common/services/DeepSeekService'
import { PomodoroService } from '../common/services/PomodoroService'
import { ProfileAnalyzerService } from '../common/services/ProfileAnalyzerService'
import { StorageManager } from '../common/utils/StorageManager'
import { Logger } from '../common/utils/Logger'
import { safeAreaHelper } from '../common/utils/SafeAreaHelper'
import { DimensConstants } from '../common/constants/DimensConstants'
import { CommonConstants } from '../common/constants/CommonConstants'
import { MAX_INPUT_LENGTH } from '../model/AppConstants'
import { formatEstimatedDuration } from '../common/utils/SafeUtils'
```

```
import { routeVM } from '../viewmodel/RoutePageViewModel'
import { settingsVM } from '../viewmodel/SettingsViewModel'
import { homeVM } from '../viewmodel/HomeViewModel'
import { profileVM } from '../viewmodel/ProfileViewModel'
import { ROUTE_NAME, navigateTo, pushNavPath } from '../common/RouterManager'
import { ChatMessageDataSource } from '../common/datasource/TaskDataSource'
import { backgroundTaskManager } from '@kit.BackgroundTasksKit'
import { CommonTitleBar } from '../common/components/CommonTitleBar'
import { AppActionSheet, AppActionSheetItem } from '../common/components/AppActionSheet'
import { ChatMessageItem, AiScenarioChipSection, AiLoginRequiredSection, AiGuideSection, AiPlanSideSection
} from '../common/components/ai/AIChatComponents'
// 用命名空间方式引用 DelaySuspendInfo
type DelaySuspendInfo = backgroundTaskManager.DelaySuspendInfo
// 平板横屏双栏阈值（与全局 lg 断点一致）。
// 注意：AI 页双栏以页面实测宽度（onAreaChange）判定，不依赖全局断点监听——
// 实测是物理事实，不存在"监听没绑上所以平板永远走手机布局"的失效路径。
const WIDE_PANE_MIN_WIDTH: number = CommonConstants.BREAKPOINT_LG
/**
 * AI 对话历史中的 plan 快照（用于构造 history 时序列化）
 * 刻意只保留任务名/优先级/截止日：完整任务对象（含 steps）会让多轮对话的
 * history 膨胀到数千 token，显著拖慢响应；AI 上下文只需要"之前提过哪些任务"
 */
interface AIHistoryPlanTask {
  taskName: string
  priority: string
  deadline: number
}
interface AIHistoryPlanSnapshot {
  goalName: string
  tasks: Array<AIHistoryPlanTask>
}
/** 从完整任务对象提取历史快照所需的精简字段 */
function buildHistoryPlanTask(t: AiGeneratedTask): AIHistoryPlanTask {
  const item: AIHistoryPlanTask = {
    taskName: t.taskName,
    priority: String(t.priority),
    deadline: t.deadline
  }
  return item
}
/* ============================================================
 * 根组件：AIPage — AI 智能拆解页
 * ============================================================ */
/**
 * AI 智能拆解页
 *
 * 职责：
 *   - 持有全部 @Local 页面级状态
 *   - 执行全部业务方法（网络请求、数据持久化、路由跳转、弹窗）
 *   - 组装子组件（build 方法仅含条件判断 + 子组件标签）
```

```
 */
@ComponentV2
export struct AIPage {
  /** 用户输入文本（底部输入框） */
  @Local inputText: string = ''
  /** 是否正在等待 AI 回复（用于禁用发送按钮） */
  @Local isLoading: boolean = false
  /**
   * Step 2 重构：对话流消息列表（仅内存，绝不入库）
   * 红线：chatMessages 仅存在于 @Local，绝不写入 routeVM.goals 或 CloudSyncService.SyncDataBundle
   *       落库动作仅通过 handleConfirmPlan → createGoalFromAi + routeVM.addGoal
   */
  @Local chatMessages: Array<ChatMessage> = []
  /**
   * LazyForEach 数据源：与 chatMessages 同步
   * - chatMessages 数组引用变更时通过 @Monitor 同步
   * - chatMessages[i] 属性变更由 @Trace 直接驱动子组件 @Param 刷新
   */
  @Local chatDataSource: ChatMessageDataSource = new ChatMessageDataSource()
  /** 已展开的任务步骤索引 Map<messageId_taskIndex, true>（替代旧 Set，支持多消息隔离） */
  @Local expandedTaskIndices: Map<string, boolean> = new Map()
  /** lg 侧栏任务卡独立展开状态（不与消息流共享，避免互相影响） */
  @Local sideExpandedTaskIndices: Map<string, boolean> = new Map()
  @Local requestSeq: number = 0
  /**
   * 监听 chatMessages 引用变更，同步至 LazyForEach 数据源
   * - 仅数组引用变更触发（如 push 后重新赋值）
   * - 数组元素属性变更（如 isConfirmed）由 @Trace 直接驱动，无需此处同步
   */
  @Monitor('chatMessages')
  onChatMessagesChange(): void {
    this.chatDataSource.updateAll(this.chatMessages)
  }
  private bgRequestInfo: DelaySuspendInfo | null = null
  /**
   * 深度修复：组件存活标记
   * 防止页面已销毁/未挂载时仍被回调污染状态。
   */
  private isAttached: boolean = false
  /** ArkTS 重构：场景芯片按压态（替代废弃的 .stateStyles） */
  @Local pressedChipId: string = ''
  /** ArkTS 重构：任务卡按压态（替代废弃的 .stateStyles） */
  @Local pressedTaskCardId: string = ''
  /**
   * 登录态拦截：改为**在 build() 中直接派生**（不再用 @Local 快照）。
   *
   * 原实现只在 aboutToAppear / onPageShow 里赋值。游客是在应用内点"去登录"拉起 LoginPage，
   * 而 LoginPage 是压在 Navigation 栈上的 NavDestination，MainTab 及其 Tabs 并未真正 onPageHide，
   * 返回时 onPageShow 不保证触发 → 快照不更新 → 出现"已经登录了，AI 页却还显示去登录"。
   *
```

```
 * 现统一读 `settingsVM.userProfile.isLoggedIn`（SettingsViewModel @ObservedV2 → userProfile @Trace
 * → isLoggedIn @Trace 的完整响应链），登录成功即自动重渲染当前所有读取该值的界面。
 * 消费点见 build() 中的 AiLoginRequiredSection 分支。
 */
/** 编辑任务弹窗状态 */
@Local showEditTaskDialog: boolean = false
/** 统一操作面板状态 */
@Local showActionSheet: boolean = false
@Local actionSheetTitle: string = ''
@Local actionSheetItems: AppActionSheetItem[] = []
/** 当前编辑的任务所属消息 ID */
@Local editingMessageId: string = ''
/** 当前编辑的任务 ID */
@Local editingTaskId: string = ''
/** 编辑中的任务名称 */
@Local editTaskName: string = ''
/** 编辑中的预估时间（分钟） */
@Local editTaskTime: string = ''
/** 对话列表 Scroller 控制器（用于自动滚动到底部 + 悬浮按钮） */
private scroller: Scroller = new Scroller()
/** 是否显示"滚动到底部"悬浮按钮（仅当列表偏离底部时显示） */
@Local showScrollToBottom: boolean = false
/** DeepSeek 服务单例 */
private service: DeepSeekService = DeepSeekService.getInstance()
/** 打字机逐字显示定时器（-1 = 未在进行） */
private revealTimerId: number = -1
/** 正在逐字显示的消息 ID（空串 = 未在进行） */
private revealMsgId: string = ''
/**
 * Step 2 重构：切换指定消息内任务的步骤展开状态
 * @param messageId - 消息 ID
 * @param taskIndex - 任务在该消息内的索引
 */
private toggleExpand(messageId: string, taskIndex: number): void {
  const key: string = messageId + '_' + taskIndex.toString()
  const newMap: Map<string, boolean> = new Map(this.expandedTaskIndices)
  if (newMap.has(key)) {
    newMap.delete(key)
  } else {
    newMap.set(key, true)
  }
  this.expandedTaskIndices = newMap
}
/** lg 侧栏：切换侧栏任务卡的步骤展开状态（独立 Map，不影响消息流） */
private toggleSideExpand(messageId: string, taskIndex: number): void {
  const key: string = messageId + '_' + taskIndex.toString()
  const newMap: Map<string, boolean> = new Map(this.sideExpandedTaskIndices)
  if (newMap.has(key)) {
    newMap.delete(key)
  } else {
```

```
      newMap.set(key, true)
    }
    this.sideExpandedTaskIndices = newMap
  }
  /** lg 侧栏：最新一条已生成的拆解方案消息（assistant 且携带任务且非流式） */
  @Computed
  get latestPlanMessage(): ChatMessage | null {
    let latest: ChatMessage | null = null
    for (let i = 0; i < this.chatMessages.length; i++) {
      const msg: ChatMessage = this.chatMessages[i]
      if (this.isPlanMessage(msg)) {
        latest = msg
      }
    }
    return latest
  }
  /**
   * 是否存在多条方案（≥2 条）。
   * 右栏恒为最新一份、左栏保留历史方案卡片；仅当确有历史时才在右栏标「最新」，
   * 避免只有一条方案时出现无意义的标识。
   */
  @Computed
  get hasPlanHistory(): boolean {
    let count: number = 0
    for (let i: number = 0; i < this.chatMessages.length; i++) {
      if (this.isPlanMessage(this.chatMessages[i])) {
        count++
      }
    }
    return count > 1
  }
  /** 实测内容区宽度（onAreaChange 上报），平板双栏的唯一判定来源 */
  @Local contentWidth: number = 0
  /** 是否宽屏双栏布局：页面实测宽度 ≥ lg 阈值（840vp） */
  private isWidePane(): boolean {
    return this.contentWidth >= WIDE_PANE_MIN_WIDTH
  }
  /** 方案消息判定：assistant 且携带任务且非流式（聊天流隐藏与右栏展示共用同一谓词） */
  private isPlanMessage(msg: ChatMessage): boolean {
    return msg.role === 'assistant' && msg.tasks.length > 0 && !msg.isStreaming
  }
  /**
   * 宽屏双栏：仅「最新一条」方案的任务卡片移到右侧方案栏；
   * 历史方案的卡片仍留在对话流中（文字 + 卡片 + 确认按钮都在），
   * 便于用户回看、对比，或觉得新方案不合适时改用旧方案。
   */
  private shouldHidePlanCards(msg: ChatMessage): boolean {
    if (!this.isWidePane() || msg.role !== 'assistant' || msg.tasks.length === 0) {
      return false
    }
```

```
    const latest: ChatMessage | null = this.latestPlanMessage
    return latest !== null && latest.id === msg.id
  }
  /** 内容列限宽：宽屏双栏铺满（聊天 layoutWeight + 右侧方案栏），手机保持限宽居中 */
  private getContentConstraint(): ConstraintSizeOptions {
    if (this.isWidePane()) {
      const full: ConstraintSizeOptions = { maxWidth: '100%' }
      return full
    }
    const capped: ConstraintSizeOptions = { maxWidth: DimensConstants.CONTENT_MAX_WIDTH }
    return capped
  }
  /**
   * Step 2 重构：打开编辑任务弹窗（微调）
   * 用当前任务数据填充编辑表单
   *
   * @param messageId - 任务所属消息 ID
   * @param task - 要编辑的 AI 生成任务
   */
  private handleEditTask(messageId: string, task: AiGeneratedTask): void {
    if (!task || !task.id) return
    this.editingMessageId = messageId
    this.editingTaskId = task.id
    this.editTaskName = task.taskName
    this.editTaskTime = task.estimatedTime > 0 ? task.estimatedTime.toString() : ''
    this.showEditTaskDialog = true
    Logger.info('AIPage', 'handleEditTask: msg=' + messageId + ', taskId=' + task.id)
  }
  /**
   * 保存编辑结果
   * Step 2 重构：在 chatMessages 中查找并更新对应任务
   */
  private saveEditTask(): void {
    if (!this.editingTaskId || this.editingTaskId.length === 0) return
    const trimmedName: string = this.editTaskName.trim()
    if (trimmedName.length === 0) {
      promptAction.showToast({
        message: getContext(this).resourceManager.getStringSync($r('app.string.toast_task_name_required').id
),
        duration: 1500
      })
      return
    }
    const timeNum: number = parseInt(this.editTaskTime, 10)
    const safeTime: number = isNaN(timeNum) || timeNum < 1 ? 1 : Math.floor(timeNum)
    // Step 2 重构：在 chatMessages 中查找任务并更新（@Trace 触发响应式刷新）
    const task: AiGeneratedTask | null = this.findTaskInMessages(this.editingMessageId, this.editingTaskId)
    if (task === null) {
      this.showEditTaskDialog = false
      return
```

```
    }
    task.taskName = trimmedName
    task.estimatedTime = safeTime
    // 触发 chatMessages 响应式刷新（重新赋值数组）
    this.chatMessages = [...this.chatMessages]
    this.showEditTaskDialog = false
    Logger.info('AIPage', 'saveEditTask: msg=' + this.editingMessageId +
      ', taskId=' + this.editingTaskId +
      ', name=' + trimmedName + ', time=' + safeTime.toString())
  }
  /**
   * Step 2 重构：删除 AI 生成任务（带确认弹窗）
   *
   * @param messageId - 任务所属消息 ID
   * @param task - 要删除的任务
   */
  private handleDeleteTask(messageId: string, task: AiGeneratedTask): void {
    if (!task || !task.id) return
    // 统一 ActionSheet：替代系统 showDialog 兜底样式
    this.actionSheetTitle = getContext(this).resourceManager.getStringSync($r('app.string.ai_delete_confirm_
message').id)
    this.actionSheetItems = [
      {
        label: getContext(this).resourceManager.getStringSync($r('app.string.btn_confirm_delete').id),
        danger: true,
        action: (): void => {
          if (!this.isAttached) return
          this.doDeleteTask(messageId, task.id)
        }
      }
    ]
    this.showActionSheet = true
  }
  /**
   * 任务卡片「  」更多操作：弹出操作面板（微调 / 删除）
   * 复用 handleEditTask / handleDeleteTask（后者自身会再弹一次确认面板）
   */
  private handleTaskMoreActions(messageId: string, task: AiGeneratedTask): void {
    if (!task || !task.id) return
    const rm = getContext(this).resourceManager
    this.actionSheetTitle = task.taskName
    this.actionSheetItems = [
      {
        label: rm.getStringSync($r('app.string.ai_edit_task').id),
        action: (): void => {
          if (!this.isAttached) return
          this.handleEditTask(messageId, task)
        }
      },
      {
```

```
        label: rm.getStringSync($r('app.string.ai_delete_task').id),
        danger: true,
        action: (): void => {
          if (!this.isAttached) return
          this.handleDeleteTask(messageId, task)
        }
      }
    ]
    this.showActionSheet = true
  }
  /** lg 侧栏：一键展开 / 收起该方案下全部任务的步骤 */
  private setSideExpandAll(messageId: string, expand: boolean): void {
    let taskCount: number = 0
    for (let i: number = 0; i < this.chatMessages.length; i++) {
      if (this.chatMessages[i].id === messageId) {
        taskCount = this.chatMessages[i].tasks.length
        break
      }
    }
    const newMap: Map<string, boolean> = new Map(this.sideExpandedTaskIndices)
    for (let i: number = 0; i < taskCount; i++) {
      const key: string = messageId + '_' + i.toString()
      if (expand) {
        newMap.set(key, true)
      } else {
        newMap.delete(key)
      }
    }
    this.sideExpandedTaskIndices = newMap
  }
  /**
   * Step 2 重构：实际执行删除操作
   * 从指定消息的 tasks 数组中移除任务
   *
   * @param messageId - 消息 ID
   * @param taskId - 要删除的任务 ID
   */
  private doDeleteTask(messageId: string, taskId: string): void {
    if (!messageId || !taskId) return
    const msgIndex: number = this.findMessageIndexById(messageId)
    if (msgIndex < 0) return
    const msg: ChatMessage = this.chatMessages[msgIndex]
    const initialLen: number = msg.tasks.length
    const newList: AiGeneratedTask[] = []
    for (let i: number = 0; i < msg.tasks.length; i++) {
      if (msg.tasks[i].id !== taskId) {
        newList.push(msg.tasks[i])
      }
    }
    if (newList.length === initialLen) return
```

```
    // V2 响应式：ChatMessage 已是 @ObservedV2 class，直接修改 @Trace tasks 即可触发刷新
    msg.tasks = newList
    Logger.info('AIPage', 'doDeleteTask: msg=' + messageId + ', taskId=' + taskId +
      ', remaining=' + newList.length.toString())
  }
  /**
   * Step 2 重构：根据消息 ID 查找 chatMessages 中的索引
   */
  private findMessageIndexById(messageId: string): number {
    for (let i: number = 0; i < this.chatMessages.length; i++) {
      if (this.chatMessages[i].id === messageId) return i
    }
    return -1
  }
  /**
   * Step 2 重构：在指定消息内根据任务 ID 查找任务（返回引用，可直接修改 @Trace 属性）
   */
  private findTaskInMessages(messageId: string, taskId: string): AiGeneratedTask | null {
    const msgIndex: number = this.findMessageIndexById(messageId)
    if (msgIndex < 0) return null
    const msg: ChatMessage = this.chatMessages[msgIndex]
    for (let i: number = 0; i < msg.tasks.length; i++) {
      if (msg.tasks[i].id === taskId) return msg.tasks[i]
    }
    return null
  }
  /**
   * Step 2 重构：发送消息（对话流主入口）
   *
   * 流程：
   *   1. 用户输入构造为 ChatMessage(role=user) 推入 chatMessages
   *   2. 构造空的 AI ChatMessage(isStreaming=true) 推入 chatMessages
   *   3. 将历史 chatMessages（排除 isStreaming 空消息）映射为 history 传给 service
   *   4. 收到 AIParseResult 后，更新刚才那个 AI ChatMessage 的 text/tasks/goalName
   *
   * 红线：chatMessages 仅内存，绝不入库；落库仅通过 handleConfirmPlan
   */
  /**
   * 发送消息：构造用户消息 + AI 占位消息，调用 DeepSeek 生成拆解方案
   */
  private async handleSendMessage(): Promise<void> {
    // [PO-privacy] 云端同步关闭时拦截 AI 网络请求
    if (!settingsVM.settings.syncEnabled) {
      promptAction.showToast({
        message: $r('app.string.ai_cloud_sync_required_toast'),
        duration: 2000
      })
      Logger.info('AIPage', 'handleSendMessage blocked: cloud sync disabled')
      return
    }
```

```
    const text: string = this.inputText.trim()
    if (text.length === 0) {
      promptAction.showToast({ message: getContext(this).resourceManager.getStringSync($r('app.string.toast_
input_required').id) })
      return
    }
    if (this.isLoading) {
      return
    }
    // 深度修复：请求序列号（防止旧回调污染新状态）
    this.requestSeq += 1
    const localSeq: number = this.requestSeq
    // 1. 构造用户消息并推入 chatMessages
    const userMsgId: string = 'msg_user_' + Date.now().toString()
    const userMsg: ChatMessage = new ChatMessage(userMsgId, 'user', text, [], '', false, false)
    this.chatMessages = [...this.chatMessages, userMsg]
    // 2. 构造空的 AI 消息（isStreaming=true，UI 显示 loading）
    const aiMsgId: string = 'msg_ai_' + Date.now().toString()
    const aiMsg: ChatMessage = new ChatMessage(aiMsgId, 'assistant', '', [], '', true, false)
    this.chatMessages = [...this.chatMessages, aiMsg]
    // 自动滚动到底部（展示最新的 loading 气泡）
    this.scrollToBottom()
    // 3. 清空输入框，置 loading
    this.inputText = ''
    this.isLoading = true
    // 4. 申请后台长时任务
    try {
      this.bgRequestInfo = backgroundTaskManager.requestSuspendDelay(
        'AIGenerate',
        (): void => {
          Logger.warn('AIPage', 'background suspend delay expired, request may be killed')
        }
      )
    } catch (e) {
      Logger.warn('AIPage', 'requestSuspendDelay failed: ' + (e as Error).message)
    }
    Logger.info('AIPage', 'handleSendMessage: seq=' + localSeq.toString() +
      ', userMsgId=' + userMsgId + ', aiMsgId=' + aiMsgId)
    try {
      // 5. 构造多轮对话 history（过滤掉当前 isStreaming 的空 AI 消息，只取 role + text）
      // 注：chatMessages 中已包含本次刚推入的 user 消息 + isStreaming=true 的 AI 空消息
      //      history 应只包含"本次之前的"完整对话，故排除最后两条（user + streaming AI）
      // 修复：AI 消息需补上 <plan> 标签，让 AI 能看到自己之前拆解的任务结构（避免失忆）
      // 限制：最多保留最近 16 条（8 轮对话），防止极端情况下 token 超限
      const history: Array<DeepSeekMessage> = []
      const excludeTail: number = 2 // 排除刚推入的 user + streaming AI
      const maxHistory: number = 16 // 8 轮对话 = 16 条消息
      const historyEnd: number = Math.max(0, this.chatMessages.length - excludeTail)
      const startIdx: number = Math.max(0, historyEnd - maxHistory)
      // 上次被跳过的 user 消息（用于补位：当 error assistant 被移除时，其前的 user 也应跳过）
```

```
    let lastSkippedUser: boolean = false
    for (let i: number = startIdx; i < historyEnd; i++) {
      const msg: ChatMessage = this.chatMessages[i]
      // 跳过空消息或 streaming 中的消息
      if (msg.text.length === 0 || msg.isStreaming) continue
      // 跳过 assistant 错误消息（防止"AI 生成失败"等文本污染 DeepSeek 上下文导致 400）
      if (msg.role === 'assistant' && msg.tasks.length === 0 && !msg.isConfirmed) {
        if (msg.text.indexOf('失败') >= 0 || msg.text.indexOf('错误') >= 0 ||
          msg.text.indexOf('重试') >= 0 || msg.text.indexOf('稍后') >= 0 ||
          msg.text.indexOf('异常') >= 0 || msg.text.indexOf('HTTP') >= 0) {
          // 标记：上一次推入的 user 消息需要被移除（避免 user → (error) → user 连续 user 违规）
          lastSkippedUser = true
          Logger.info('AIPage', 'history: skip error assistant msg at index ' + i.toString() +
            ', text=' + msg.text.substring(0, 50))
          continue
        }
      }
      // 如果上一个 error assistant 被跳过，其前面的 user 消息也应跳过
      if (lastSkippedUser && msg.role === 'user') {
        lastSkippedUser = false
        Logger.info('AIPage', 'history: skip user msg after error assistant, id=' + msg.id)
        continue
      }
      lastSkippedUser = false
      let content: string = msg.text
      // AI 消息且携带任务：补上精简版 <plan> 标签，让 AI 能看到自己之前拆解过什么（不传完整结构，控制
token）
      if (msg.role === 'assistant' && msg.tasks.length > 0) {
        const planTasks: Array<AIHistoryPlanTask> = []
        for (let i: number = 0; i < msg.tasks.length; i++) {
          planTasks.push(buildHistoryPlanTask(msg.tasks[i]))
        }
        const planObj: AIHistoryPlanSnapshot = {
          goalName: msg.goalName,
          tasks: planTasks
        }
        content = msg.text + '\n<plan>\n' + JSON.stringify(planObj) + '\n</plan>'
      }
      const historyMsg: DeepSeekMessage = {
        role: msg.role,
        content: content
      }
      history.push(historyMsg)
    }
    // 边界修复：如果 error assistant 是 history 窗口最后一条，其前的 user 仍在 history 末尾，
    // 后续 buildRequest 会追加 currentUserInput，导致连续两个 user 违规
    if (lastSkippedUser && history.length > 0 &&
      history[history.length - 1].role === 'user') {
      Logger.info('AIPage', 'history: pop trailing user after error assistant at boundary')
      history.pop()
```

```
      }
      Logger.info('AIPage', 'handleSendMessage: history 构造完成, count=' + history.length.toString() +
        ', startIdx=' + startIdx.toString() + ', historyEnd=' + historyEnd.toString())
      // 6. 获取用户画像摘要（fire-and-forget 风格，失败不阻断 AI 调用）
      let profilePrompt: string = ''
      try {
        profilePrompt = await ProfileAnalyzerService.buildProfilePrompt(
          routeVM, homeVM, PomodoroService.getInstance())
        if (profilePrompt.length > 0) {
          Logger.info('AIPage', 'profilePrompt: ' + profilePrompt)
        }
      } catch (e) {
        Logger.warn('AIPage', 'buildProfilePrompt failed, skip: ' + (e as Error).message)
      }
      // 7. 调用 DeepSeekService（传入 history + 当前用户输入 + 画像，service 内部拼
system+profile+history+user）
      //    M2 云函数中转：移除 apiKey 参数，由云函数环境变量持有密钥
      const result: AIParseResult = await this.service.generateTasks(
        history, text, profilePrompt, [])
      // 深度修复：校验 seq + 组件存活
      if (localSeq !== this.requestSeq) {
        Logger.info('AIPage', 'stale response, seq=' + localSeq.toString() + ', discarded')
        return
      }
      if (!this.isAttached) {
        Logger.info('AIPage', 'component detached, discarded')
        return
      }
      // 6. 更新刚才那个 AI 消息
      const msgIndex: number = this.findMessageIndexById(aiMsgId)
      if (msgIndex < 0) {
        Logger.warn('AIPage', 'ai message not found after await: ' + aiMsgId)
        return
      }
      // V2 响应式：ChatMessage 已是 @ObservedV2 class，直接修改 @Trace 属性即可触发 @Param 刷新
      // 无需构造新对象替换数组元素，@Trace 会自动通知 @Param 监听器
      const aiMsg2: ChatMessage = this.chatMessages[msgIndex]
      aiMsg2.tasks = result.tasks
      aiMsg2.goalName = result.goalName
      // 打字机逐字显示（伪流式）：文本从空逐段追加，任务卡与确认按钮在显示完成后出现
      this.startTypewriterReveal(aiMsgId, result.text)
      Logger.info('AIPage', 'handleSendMessage success, aiMsgId=' + aiMsgId +
        ', textLen=' + result.text.length.toString() +
        ', goalName=' + result.goalName +
        ', tasks=' + result.tasks.length.toString())
    } catch (e) {
      if (localSeq !== this.requestSeq) {
        Logger.info('AIPage', 'stale error, seq=' + localSeq.toString() + ', discarded')
        return
      }
```

```
      if (!this.isAttached) return
      const err: Error = e as Error
      Logger.error('AIPage', 'handleSendMessage failed: ' + err.message)
      // 更新 AI 消息为错误态（在气泡内显示错误文本）
      const msgIndex: number = this.findMessageIndexById(aiMsgId)
      if (msgIndex >= 0) {
        // V2 响应式：ChatMessage 已是 @ObservedV2 class，直接改 @Trace 属性
        const errMsgObj: ChatMessage = this.chatMessages[msgIndex]
        errMsgObj.text = getContext(this).resourceManager.getStringSync($r('app.string.error_generic_toast')
.id)
        errMsgObj.isStreaming = false
      }
      // M2 云函数中转：错误提示统一化，不再区分 API Key 失效
      const errMsg: string = err.message
      promptAction.showToast({ message: getContext(this).resourceManager.getStringSync($r('app.string.error_
generic_toast').id), duration: 2000 })
      Logger.error('AIPage', 'handleSendMessage AI failed: ' + errMsg)
    } finally {
      // 修复：finally 必须无条件重置 isLoading，否则任何边界情况都会导致永久卡死
      // （原条件 localSeq === this.requestSeq 在 requestSeq 被改时会跳过重置，导致下次发送被拦截）
      this.isLoading = false
      this.releaseBackgroundTask()
    }
  }
  /**
   * 举报 AI 生成内容（合规要求：AIGC 应用需提供便捷举报入口）
   * 复用反馈页，以 report 模式打开（标题/分类/文案切换为举报语义）
   *
   * @param messageId - 被举报的 AI 消息 ID（仅用于日志溯源，内容由用户在举报页描述）
   */
  private handleReportAiContent(messageId: string): void {
    Logger.info('AIPage', 'handleReportAiContent: open report page for message=' + messageId)
    const param: FeedbackRouteNavParam = { mode: 'report' }
    pushNavPath(ROUTE_NAME.FEEDBACK, param)
  }
  /**
   * Step 2 重构：确认使用此方案（加入路线）
   *
   * 红线（绝对不可破坏）：
   *   - 仅通过 createGoalFromAi + routeVM.addGoal 落库
   *   - 不直接操作 routeVM.goals 或 CloudSyncService
   *   - chatMessages 仅更新 isConfirmed 标记（内存态）
   *
   * @param messageId - AI 消息 ID
   */
  private async handleConfirmPlan(messageId: string): Promise<void> {
    const msgIndex: number = this.findMessageIndexById(messageId)
    if (msgIndex < 0) {
      Logger.warn('AIPage', 'handleConfirmPlan: message not found: ' + messageId)
      return
```

```
    }
    const msg: ChatMessage = this.chatMessages[msgIndex]
    if (msg.isConfirmed) {
      Logger.info('AIPage', 'handleConfirmPlan: already confirmed: ' + messageId)
      return
    }
    if (msg.tasks.length === 0) {
      promptAction.showToast({ message: getContext(this).resourceManager.getStringSync($r('app.string.toast_
no_tasks_to_add').id) })
      return
    }
    // goalName 转义（去除换行/截断），优先用 AI 返回的 goalName，缺失时从用户最近一条消息截取
    let rawGoalName: string = msg.goalName.trim()
    if (rawGoalName.length === 0) {
      rawGoalName = this.findLatestUserTextBefore(messageId)
    }
    const goalName: string = rawGoalName.length > 0 ?
      rawGoalName.replace(/[\r\n]+/g, ' ').substring(0, 50) :
      getContext(this).resourceManager.getStringSync($r('app.string.ai_default_goal_name').id)
    // 红线：通过 AiGoalFactory + routeVM.addGoal 单一入库路径
    const newGoal: GoalCardItem = createGoalFromAi(msg.tasks, goalName, getContext(this))
    try {
      routeVM.addGoal(newGoal)
      Logger.info('AIPage', 'handleConfirmPlan: routeVM.addGoal success, goalId=' + newGoal.id +
        ', name=' + newGoal.name + ', tasks=' + newGoal.tasks.length.toString())
    } catch (e) {
      const err: Error = e as Error
      Logger.error('AIPage', 'handleConfirmPlan: routeVM.addGoal FAILED: ' + err.message)
      promptAction.showToast({
        message: getContext(this).resourceManager.getStringSync($r('app.string.toast_goal_add_failed_prefix'
).id) +
          getContext(this).resourceManager.getStringSync($r('app.string.error_generic_toast').id),
        duration: 2000
      })
      return
    }
    // 标记该消息为已确认（仅内存态，刷新 UI 为"已加入路线  "）
    // V2 响应式：ChatMessage 已是 @ObservedV2 class，直接改 @Trace isConfirmed
    this.chatMessages[msgIndex].isConfirmed = true
    // 首次使用场景引导开启系统通知：确认方案意味着产生了带 deadline 的任务，
    // 不开通知则截止提醒会静默失败。fire-and-forget，不阻断主流程
    profileVM.ensureNotificationEnabled().catch((e: Error): void => {
      Logger.warn('AIPage', 'ensureNotificationEnabled failed: ' + e.message)
    })
    promptAction.showToast({
      message: getContext(this).resourceManager.getStringSync($r('app.string.toast_goal_added_prefix').id) +
        newGoal.name +
        getContext(this).resourceManager.getStringSync($r('app.string.toast_goal_added_task_infix').id) +
        newGoal.tasks.length.toString() +
        getContext(this).resourceManager.getStringSync($r('app.string.toast_goal_added_task_suffix').id),
```

```
      duration: 2000
    })
    // 自动将高优先级任务同步到首页待办清单（失败不阻断目标创建流程）
    try {
      const allTasks: AiGeneratedTask[] = newGoal.tasks
      const highPriorityTasks: AiGeneratedTask[] = []
      for (let i: number = 0; i < allTasks.length; i++) {
        if (allTasks[i].priority === TaskPriority.HIGH) {
          highPriorityTasks.push(allTasks[i])
        }
      }
      // 筛选策略：优先高优先级，不足 2 个则从原数组取前 2 个补足
      const MAX_SYNC_COUNT: number = 2
      let selectedTasks: AiGeneratedTask[] = []
      if (highPriorityTasks.length >= MAX_SYNC_COUNT) {
        selectedTasks = highPriorityTasks.slice(0, MAX_SYNC_COUNT)
      } else {
        selectedTasks = highPriorityTasks.slice()
        for (let i: number = 0; i < allTasks.length && selectedTasks.length < MAX_SYNC_COUNT; i++) {
          const t: AiGeneratedTask = allTasks[i]
          if (t.priority !== TaskPriority.HIGH) {
            selectedTasks.push(t)
          }
        }
      }
      // 逐个构造 HomeTodoItem 并加入首页待办
      let addedCount: number = 0
      for (let i: number = 0; i < selectedTasks.length; i++) {
        const aiTask: AiGeneratedTask = selectedTasks[i]
        const todoId: string = 'home_ai_' + newGoal.id + '_' + aiTask.id
        const durationStr: string = formatEstimatedDuration(aiTask.estimatedTime > 0 ? aiTask.estimatedTime
: 30, getContext(this).resourceManager)
        const todo: HomeTodoItem = buildHomeTodoItem(
          todoId,
          aiTask.taskName,
          newGoal.name,
          durationStr,
          aiTask.priority,
          false,
          Date.now(),
          aiTask.id,
          newGoal.id,
          0,
          0,
          aiTask.deadline
        )
        const ok: boolean = homeVM.addExtractedTask(todo)
        if (ok) {
          addedCount++
        }
```

```
      }
      if (addedCount > 0) {
        promptAction.showToast({
          message: getContext(this).resourceManager.getStringSync($r('app.string.ai_added_tasks_to_todo_pref
ix').id) +
            addedCount.toString() +
            getContext(this).resourceManager.getStringSync($r('app.string.ai_added_tasks_to_todo_suffix').id
),
          duration: 2000
        })
        Logger.info('AIPage', 'handleConfirmPlan: synced ' + addedCount.toString() +
          ' tasks to home todo, goalId=' + newGoal.id)
      }
    } catch (e) {
      Logger.warn('AIPage', 'handleConfirmPlan: sync tasks to home todo failed: ' +
        (e as Error).message + ', goalId=' + newGoal.id)
    }
  }
  /**
   * Step 2 重构：查找指定 AI 消息之前最近一条用户消息文本（用于 goalName fallback）
   */
  private findLatestUserTextBefore(messageId: string): string {
    const msgIndex: number = this.findMessageIndexById(messageId)
    if (msgIndex < 0) return ''
    // 从 msgIndex-1 倒序查找 role=user 的消息
    for (let i: number = msgIndex - 1; i >= 0; i--) {
      if (this.chatMessages[i].role === 'user') {
        return this.chatMessages[i].text
      }
    }
    return ''
  }
  /**
   * 打字机逐字显示（伪流式）：拿到完整回复后按节奏追加到 text
   * - 观感为"AI 正在打字"，替代一次性全文出现
   * - 期间 isStreaming 保持 true，任务卡与确认按钮延后出现
   * - 若上一条消息还在逐字显示中，先补全上一条再开始新的
   */
  private startTypewriterReveal(msgId: string, fullText: string): void {
    // 上一轮逐字显示未结束时：立即补全，保证同一时刻只有一条消息在逐字显示
    this.finishTypewriterReveal()
    const msgIndex: number = this.findMessageIndexById(msgId)
    if (msgIndex < 0) {
      return
    }
    const msg: ChatMessage = this.chatMessages[msgIndex]
    msg.fullText = fullText
    msg.text = ''
    msg.isStreaming = true
    // 总时长目标约 1.5 秒：按文本长度分片，每 24ms 追加一片
```

```
    const totalTicks: number = 60
    const chunkSize: number = Math.max(1, Math.ceil(fullText.length / totalTicks))
    this.revealMsgId = msgId
    this.revealTimerId = setInterval((): void => {
      const idx: number = this.findMessageIndexById(this.revealMsgId)
      if (idx < 0) {
        this.stopTypewriterReveal()
        return
      }
      const m: ChatMessage = this.chatMessages[idx]
      if (m.text.length >= m.fullText.length) {
        this.finishTypewriterReveal()
        return
      }
      m.text = m.fullText.substring(0, Math.min(m.fullText.length, m.text.length + chunkSize))
      this.scrollToBottom()
    }, 24) as number
  }
  /** 补全逐字显示：立即把 text 置为完整文本并结束 isStreaming 态 */
  private finishTypewriterReveal(): void {
    if (this.revealTimerId !== -1) {
      clearInterval(this.revealTimerId)
      this.revealTimerId = -1
    }
    const idx: number = this.findMessageIndexById(this.revealMsgId)
    this.revealMsgId = ''
    if (idx >= 0) {
      const m: ChatMessage = this.chatMessages[idx]
      m.text = m.fullText
      m.isStreaming = false
    }
  }
  /** 仅停止定时器（不补全文本），用于页面销毁等场景 */
  private stopTypewriterReveal(): void {
    if (this.revealTimerId !== -1) {
      clearInterval(this.revealTimerId)
      this.revealTimerId = -1
    }
    this.revealMsgId = ''
  }
  /**
   * 取消 AI 拆解请求
   */
  private handleCancel(): void {
    // 打字机逐字显示进行中：请求已返回，无网络可取消，直接补全显示
    if (this.revealTimerId !== -1) {
      this.finishTypewriterReveal()
      return
    }
    this.requestSeq += 1
```

```
    this.service.cancelRequest()
    this.isLoading = false
    this.releaseBackgroundTask()
    // Step 2 重构：将最后一条 isStreaming 的 AI 消息移除（避免气泡卡在 loading）
    if (this.chatMessages.length > 0) {
      const last: ChatMessage = this.chatMessages[this.chatMessages.length - 1]
      if (last.isStreaming) {
        this.chatMessages = this.chatMessages.slice(0, this.chatMessages.length - 1)
      }
    }
    Logger.info('AIPage', 'handleCancel: cancelled by user, seq bumped to ' + this.requestSeq.toString())
    promptAction.showToast({ message: getContext(this).resourceManager.getStringSync($r('app.string.toast_ca
ncelled').id) })
  }
  /**
   * 对话列表滚动到底部（并隐藏悬浮按钮）
   */
  private scrollToBottom(): void {
    const total: number = this.chatMessages.length
    if (total > 0) {
      this.scroller.scrollToIndex(total - 1)
    }
    this.showScrollToBottom = false
  }
  /**
   * [Fix]: 释放后台长时任务（cancelSuspendDelay）
   */
  private releaseBackgroundTask(): void {
    if (this.bgRequestInfo !== null) {
      try {
        backgroundTaskManager.cancelSuspendDelay(this.bgRequestInfo.requestId)
        Logger.info('AIPage', 'cancelSuspendDelay done')
      } catch (e) {
        Logger.warn('AIPage', 'cancelSuspendDelay failed: ' + (e as Error).message)
      }
      this.bgRequestInfo = null
    }
  }
  /**
   * Step 2 重构：页面初始化
   * - 标记组件存活
   * - 恢复输入草稿（对话流历史不持久化，仅内存）
   */
  async aboutToAppear(): Promise<void> {
    this.isAttached = true
    await this.restoreDraft()
  }
  /**
   * 深度修复：页面销毁/隐藏前
   * - 标记组件已销毁
```

```
   * - 取消进行中的请求
   * - 保存草稿（解决"无后悔药"）
   * Step 2 重构：chatMessages 不持久化（仅内存），销毁即清空
   */
  aboutToDisappear(): void {
    this.isAttached = false
    // 停止打字机逐字显示定时器（避免销毁后定时器仍操作消息）
    this.stopTypewriterReveal()
    // 卸载时如果有未完成的请求，bump seq 避免迟到回调污染
    this.requestSeq += 1
    if (this.isLoading) {
      this.service.cancelRequest()
      this.isLoading = false
    }
    // [Fix]: 页面销毁时释放后台长时任务
    this.releaseBackgroundTask()
    this.saveDraft()
  }
  /**
   * Fix: B2-5+AIPage onPageShow状态刷新
   * M2 云函数中转：移除 API Key 状态检查（Key 已存放于云函数环境变量）
   * Fix: B2-8+AIPage登录态拦截
   * 检查登录态，未登录则禁用 AI 功能
   */
  onPageShow(): void {
    // 登录态不在此处刷新：未登录拦截已改为在 build() 中直接派生自 settingsVM.userProfile.isLoggedIn，
    // 登录成功（@Trace 变更）即触发重渲染，不依赖 onPageShow 是否触发。此处仅保留日志。
    Logger.info('AIPage', 'onPageShow, isLoggedIn=' + settingsVM.userProfile.isLoggedIn.toString())
  }
  /**
   * 深度修复：恢复草稿（解决切走/返回输入丢失）
   * // Refactor: P3-6+AppStorage草稿迁移至StorageManager
   * // Refactor: P2-4+草稿操作迁移至StorageManager
   */
  private async restoreDraft(): Promise<void> {
    try {
      const draft: string = await StorageManager.loadDraft('ai_draft_text')
      if (draft.length > 0 && this.inputText.length === 0) {
        this.inputText = draft
        Logger.info('AIPage', 'restored draft, length=' + draft.length.toString())
      }
    } catch (e) {
      Logger.warn('AIPage', 'restoreDraft failed: ' + (e as Error).message)
    }
  }
  /**
   * 深度修复：保存草稿
   * 用户输入 200 字后切走，输入不丢
   * // Refactor: P2-4+草稿操作迁移至StorageManager
   */
```

```
private saveDraft(): void {
  StorageManager.saveDraft('ai_draft_text', this.inputText)
}
@Builder
EditTaskDialogBuilder(): void {
  if (this.showEditTaskDialog) {
    Column({ space: 20 }) {
      Text($r('app.string.ai_edit_task_title'))
        .fontSize(16)
        .fontWeight(FontWeight.Bold)
        .fontColor($r('app.color.text_primary'))
        .width('100%')
        .textAlign(TextAlign.Center)
      Column({ space: 12 }) {
        Text($r('app.string.ai_edit_task_name_hint'))
          .fontSize(12)
          .fontColor($r('app.color.text_secondary'))
          .width('100%')
        TextInput({
          placeholder: $r('app.string.ai_edit_task_name_hint'),
          text: this.editTaskName
        })
          .height(44)
          .fontSize(14)
          .fontColor($r('app.color.text_primary'))
          .backgroundColor($r('app.color.bg_tag'))
          .borderRadius(DimensConstants.RADIUS_M)
          .padding({ left: 12, right: 12 })
          .maxLength(100)
          .onChange((value: string): void => {
            this.editTaskName = value
          })
        Text($r('app.string.ai_edit_task_time_hint'))
          .fontSize(12)
          .fontColor($r('app.color.text_secondary'))
          .width('100%')
          .margin({ top: 8 })
        TextInput({
          placeholder: $r('app.string.ai_edit_task_time_hint'),
          text: this.editTaskTime
        })
          .height(44)
          .fontSize(14)
          .fontColor($r('app.color.text_primary'))
          .backgroundColor($r('app.color.bg_tag'))
          .borderRadius(DimensConstants.RADIUS_M)
          .padding({ left: 12, right: 12 })
          .type(InputType.Number)
          .maxLength(5)
          .onChange((value: string): void => {
```

```
            this.editTaskTime = value
          })
      }
      .width('100%')
      .alignItems(HorizontalAlign.Start)
      Row({ space: 12 }) {
        Button($r('app.string.common_cancel'))
          .type(ButtonType.Capsule)
          .layoutWeight(1)
          .height(44)
          .backgroundColor($r('app.color.bg_tag'))
          .fontColor($r('app.color.text_secondary'))
          .fontSize(14)
          .onClick((): void => {
            this.showEditTaskDialog = false
          })
        Button($r('app.string.btn_save'))
          .type(ButtonType.Capsule)
          .layoutWeight(1)
          .height(44)
          .backgroundColor($r('app.color.primary_color'))
          .fontColor($r('app.color.text_inverse'))
          .fontSize(14)
          .fontWeight(FontWeight.Medium)
          .onClick((): void => {
            this.saveEditTask()
          })
      }
      .width('100%')
      .margin({ top: 8 })
    }
    .width('85%')
    .constraintSize({ maxWidth: DimensConstants.SHEET_MAX_WIDTH })
    .padding(24)
    .backgroundColor($r('app.color.bg_card'))
    .borderRadius(DimensConstants.RADIUS_L)
  }
}
build() {
  // Step 2 重构：对话流布局
  // 编辑弹窗 - 统一用 Stack 包裹实现层级叠加
  Stack() {
    Row() {
    Column() {
      CommonTitleBar({ title: $r('app.string.page_ai'), showBack: false })
      if (!settingsVM.userProfile.isLoggedIn) {
        // 未登录拦截（响应式：登录成功后本分支自动切走，无需依赖 onPageShow 刷新）
        AiLoginRequiredSection({
          onNavigateLogin: (): void => { navigateTo(ROUTE_NAME.LOGIN) }
        })
```

```
let bindCount: number = 0
/**
 * 绑定全局断点状态（幂等，可重入）
 * 内部持有 BreakpointUtil 的注册/监听，把 mediaquery 结果同步到 breakpointState
 *
 * @param uiContext - 根页面的 UIContext（this.getUIContext()），见 BreakpointUtil.register 说明
 */
export function bindBreakpointState(uiContext: UIContext): void {
  bindCount++
  if (bindCount > 1) {
    return
  }
  boundListener = (bp: BreakpointType): void => {
    breakpointState.breakpoint = bp
    breakpointState.isTablet = bp === 'lg'
  }
  BreakpointUtil.register(uiContext)
  // 关键顺序：先挂监听再读初始值——若窗口在 register 与 addListener 之间发生变化，
  // change 事件能正常写入 breakpointState，不丢事件
  BreakpointUtil.addListener(boundListener)
  breakpointState.breakpoint = BreakpointUtil.getCurrentBreakpoint()
  breakpointState.isTablet = BreakpointUtil.isLargeScreen()
}
/**
 * 解绑全局断点状态
 * 与 bindBreakpointState 配对调用，计数归零时才真正注销 mediaquery 监听
 */
export function unbindBreakpointState(): void {
  if (bindCount === 0) {
    return
  }
  bindCount--
  if (bindCount > 0) {
    return
  }
  if (boundListener !== null) {
    BreakpointUtil.removeListener(boundListener)
    boundListener = null
  }
  BreakpointUtil.unregister()
}
/**
 * @file 断点适配工具
 * @description 基于 mediaquery 的屏幕断点监听封装，禁止写死屏幕宽度判断
 * @module common/utils
 * @date 2026-06-17
 */
import { mediaquery } from '@kit.ArkUI'
import { Logger } from './Logger'
import { CommonConstants } from '../constants/CommonConstants'
```

```
import { BreakpointType } from '../types/ApiTypes'
/**
 * 断点变化回调签名
 * @param breakpoint - 当前断点值
 */
export type BreakpointChangeListener = (breakpoint: BreakpointType) => void
/**
 * 断点适配工具类
 *
 * 强制规范：
 * - 禁止写死屏幕宽度判断（如 if (width > 800)）
 * - 必须使用 mediaquery 监听断点变化
 * - 监听器必须在 aboutToDisappear 中 off 解绑
 */
export class BreakpointUtil {
  /** 当前断点值 */
  private static currentBreakpoint: BreakpointType = 'sm'
  /** mediaquery 监听器（小屏  中屏） */
  private static mdListener: mediaquery.MediaQueryListener | null = null
  /** mediaquery 监听器（中屏  大屏） */
  private static lgListener: mediaquery.MediaQueryListener | null = null
  /** 已注册的业务回调列表 */
  private static listeners: BreakpointChangeListener[] = []
  /** 引用计数：防止重复注册/提前销毁 */
  private static refCount: number = 0
  /**
   * 注册断点监听
   * 必须在组件 aboutToAppear 中调用，并在 aboutToDisappear 中调用 unregister
   *
   * @param uiContext - 组件的 UIContext（this.getUIContext()）。
   *   必须走 UIContext.getMediaQuery()，不能裸调全局 mediaquery.matchMediaSync()：
   *   冷启动早期（MainTab.aboutToAppear）窗口尚未挂载完成，全局调用会绑到
   *   错误/不存在的窗口上，监听静默失效 → 断点永远停在 'sm'，平板横屏不触发 lg 布局。
   *   官方文档推荐 UIContext 形式（媒体查询指南 / UIContext 参考文档）。
   */
  static register(uiContext: UIContext): void {
    const tag: string = 'BreakpointUtil'
    BreakpointUtil.refCount++
    if (BreakpointUtil.refCount > 1) {
      Logger.info(tag, 'register: refCount=' + BreakpointUtil.refCount.toString() + ', already registered,
skip')
      return
    }
    try {
      // md 断点：宽度 >= 600vp
      BreakpointUtil.mdListener = uiContext.getMediaQuery()
        .matchMediaSync(`(min-width: ${CommonConstants.BREAKPOINT_MD}vp)`)
      BreakpointUtil.mdListener.on('change', (_result: mediaquery.MediaQueryResult): void => {
        BreakpointUtil.updateBreakpoint()
      })
```

```
      // lg 断点：宽度 >= 840vp
      BreakpointUtil.lgListener = uiContext.getMediaQuery()
        .matchMediaSync(`(min-width: ${CommonConstants.BREAKPOINT_LG}vp)`)
      BreakpointUtil.lgListener.on('change', (_result: mediaquery.MediaQueryResult): void => {
        BreakpointUtil.updateBreakpoint()
      })
      // 初始值：根据当前匹配状态推断
      BreakpointUtil.updateBreakpoint()
      Logger.info(tag, 'register success, initial={0}', BreakpointUtil.currentBreakpoint)
    } catch (error) {
      Logger.error(tag, 'register failed, msg={0}', (error as Error).message)
    }
  }
  /**
   * 联动推导当前断点（唯一写入点）
   * 两个 mediaquery 监听器的 change 事件会独立触发，且宽度 < 600vp 时 lg 监听器
   * 也会以 matches=false 触发——若各自独立写值，lg 的 false 会把正确的 sm 覆盖成 md
   * （表现为手机竖屏页面被渲染成平板双列布局）。必须每次用两个监听器的 matches
   * 状态联合推导：lg 匹配 → lg；否则 md 匹配 → md；否则 sm。
   */
  private static updateBreakpoint(): void {
    const lgMatches: boolean = BreakpointUtil.lgListener !== null && BreakpointUtil.lgListener.matches
    const mdMatches: boolean = BreakpointUtil.mdListener !== null && BreakpointUtil.mdListener.matches
    const next: BreakpointType = lgMatches ? 'lg' : (mdMatches ? 'md' : 'sm')
    if (next === BreakpointUtil.currentBreakpoint) {
      return
    }
    BreakpointUtil.currentBreakpoint = next
    Logger.info('BreakpointUtil', 'breakpoint changed to {0}', next)
    BreakpointUtil.notifyListeners()
  }
  /**
   * 注销断点监听
   * 必须在组件 aboutToDisappear 中调用，避免内存泄漏
   */
  static unregister(): void {
    const tag: string = 'BreakpointUtil'
    BreakpointUtil.refCount--
    if (BreakpointUtil.refCount > 0) {
      Logger.info(tag, 'unregister: refCount=' + BreakpointUtil.refCount.toString() + ', still in use,
skip')
      return
    }
    // 防御性保护：refCount 不应为负
    if (BreakpointUtil.refCount < 0) {
      Logger.warn(tag, 'unregister: refCount=' + BreakpointUtil.refCount.toString() + ', reset to 0')
      BreakpointUtil.refCount = 0
      return
    }
    try {
```

```
    if (BreakpointUtil.mdListener !== null) {
      BreakpointUtil.mdListener.off('change')
      BreakpointUtil.mdListener = null
    }
    if (BreakpointUtil.lgListener !== null) {
      BreakpointUtil.lgListener.off('change')
      BreakpointUtil.lgListener = null
    }
    BreakpointUtil.listeners = []
    Logger.info(tag, 'unregister success')
  } catch (error) {
    Logger.error(tag, 'unregister failed, msg={0}', (error as Error).message)
  }
}
/**
 * 添加断点变化回调
 * @param listener - 回调函数
 */
static addListener(listener: BreakpointChangeListener): void {
  BreakpointUtil.listeners.push(listener)
}
/**
 * 移除断点变化回调
 * @param listener - 回调函数
 */
static removeListener(listener: BreakpointChangeListener): void {
  const index: number = BreakpointUtil.listeners.indexOf(listener)
  if (index >= 0) {
    BreakpointUtil.listeners.splice(index, 1)
  }
}
/**
 * 获取当前断点值
 * @returns 当前断点（sm/md/lg）
 */
static getCurrentBreakpoint(): BreakpointType {
  return BreakpointUtil.currentBreakpoint
}
/**
 * 判断是否为小屏（手机竖屏）
 * @returns 是否小屏
 */
static isSmallScreen(): boolean {
  return BreakpointUtil.currentBreakpoint === 'sm'
}
/**
 * 判断是否为大屏（折叠屏展开/平板）
 * @returns 是否大屏
 */
static isLargeScreen(): boolean {
```

```
    return BreakpointUtil.currentBreakpoint === 'lg'
  }
  /**
   * 通知所有已注册的回调
   */
  private static notifyListeners(): void {
    const current: BreakpointType = BreakpointUtil.currentBreakpoint
    for (let i: number = 0; i < BreakpointUtil.listeners.length; i++) {
      try {
        BreakpointUtil.listeners[i](current)
      } catch (e) {
        // 单个回调异常不影响其他回调
      }
    }
  }
  /** 私有构造：禁止实例化 */
  private constructor() {}
}
// common/utils/BusinessError.ets
export class BusinessError extends Error {
  code: number;
  constructor(code: number, message: string) {
    super(message); // 调用父类 Error 的构造函数
    this.name = 'BusinessError';
    this.code = code;
  }
}
/**
 * @file 日志工具
 * @description 基于 hilog 封装的统一日志工具，禁止 console.log
 * @module common/utils
 * @date 2026-06-17
 */
import { hilog } from '@kit.PerformanceAnalysisKit'
import { CommonConstants } from '../constants/CommonConstants'
/**
 * 日志工具类
 *
 * 强制规范：
 * - 禁止 console.log/error/warn，必须使用 hilog
 * - 敏感信息必须脱敏（secure 方法）
 * - 统一 TAG 与 DOMAIN，便于日志过滤
 */
export class Logger {
  /** 日志域 */
  private static readonly DOMAIN: number = CommonConstants.LOG_DOMAIN
  /** 日志标签 */
  private static readonly TAG: string = CommonConstants.LOG_TAG
  /**
   * 输出 INFO 级别日志
```

```
 * @param tag - 模块标签（与全局 TAG 拼接）
 * @param format - 格式字符串
 * @param args - 格式参数（均为 public 占位，避免敏感泄露）
 */
static info(tag: string, format: string, ...args: string[]): void {
  hilog.info(Logger.DOMAIN, `${Logger.TAG}.${tag}`, '%{public}s', Logger.build(format, args))
}
/**
 * 输出 DEBUG 级别日志
 * @param tag - 模块标签
 * @param format - 格式字符串
 * @param args - 格式参数
 */
static debug(tag: string, format: string, ...args: string[]): void {
  hilog.debug(Logger.DOMAIN, `${Logger.TAG}.${tag}`, '%{private}s', Logger.build(format, args))
}
/**
 * 输出 WARN 级别日志
 * @param tag - 模块标签
 * @param format - 格式字符串
 * @param args - 格式参数
 */
static warn(tag: string, format: string, ...args: string[]): void {
  hilog.warn(Logger.DOMAIN, `${Logger.TAG}.${tag}`, '%{public}s', Logger.build(format, args))
}
/**
 * 输出 ERROR 级别日志
 * @param tag - 模块标签
 * @param format - 格式字符串
 * @param args - 格式参数
 */
static error(tag: string, format: string, ...args: string[]): void {
  hilog.error(Logger.DOMAIN, `${Logger.TAG}.${tag}`, '%{public}s', Logger.build(format, args))
}
/**
 * 输出脱敏后的敏感信息日志
 * 自动对参数进行掩码处理（保留首尾各 2 位，中间用 * 填充）
 * @param tag - 模块标签
 * @param label - 字段标签（如 "token"）
 * @param sensitive - 敏感值原文
 */
static secure(tag: string, label: string, sensitive: string): void {
  const masked: string = Logger.mask(sensitive)
  hilog.info(Logger.DOMAIN, `${Logger.TAG}.${tag}`, '%{public}s', `${label}=${masked}`)
}
/**
 * 拼接格式字符串与参数
 * @param format - 格式模板
 * @param args - 参数列表
 * @returns 拼接后的字符串
```

```
   */
  private static build(format: string, args: string[]): string {
    if (args.length === 0) {
      return format
    }
    let result: string = format
    for (let i = 0; i < args.length; i++) {
      const placeholder: string = `{${i}}`
      result = result.replace(placeholder, args[i])
    }
    return result
  }
  /**
   * 对敏感字符串进行掩码处理（全局唯一实现，各模块复用）
   * 规则：长度 ≤4 全部掩码；长度 >4 保留首尾各 2 位，中间用 **** 替换
   * @param value - 原始敏感值
   * @returns 脱敏后的字符串
   */
  static mask(value: string): string {
    if (value.length <= 4) {
      return '****'
    }
    const head: string = value.substring(0, 2)
    const tail: string = value.substring(value.length - 2)
    return `${head}****${tail}`
  }
  /** 私有构造：禁止实例化 */
  private constructor() {}
}
/** 持久化辅助类 - 统一封装防抖写入 + 异常处理，消除各 VM 重复的防抖模板代码 */
import { Logger } from './Logger'
/**
 * 持久化辅助类
 * scheduleSave 防抖写入、flush 页面退出强写、persistNow 异常兜底
 */
export class PersistHelper {
  /** 防抖定时器句柄 */
  private saveTimer: number = -1
  /** 防抖延迟（毫秒） */
  private readonly delay: number
  /** 实际持久化函数（由各VM提供） */
  private readonly persistFn: () => Promise<void>
  /** 日志标签 */
  private readonly tag: string
  /** 防抖触发时的附加回调（如云同步），flush时不触发 */
  private onScheduleFire: (() => void) | null
  /**
   * @param tag - 日志标签（通常为VM类名）
   * @param persistFn - 实际持久化函数（由各VM提供，包含序列化 + 写入）
   * @param delay - 防抖延迟毫秒数，默认200
```

```
 * @param onScheduleFire - 防抖触发时的附加回调（如云同步），flush不触发
 */
constructor(tag: string, persistFn: () => Promise<void>, delay: number = 200,
  onScheduleFire?: (() => void)) {
  this.tag = tag
  this.persistFn = persistFn
  this.delay = delay
  this.onScheduleFire = onScheduleFire !== undefined ? onScheduleFire : null
}
/**
 * 防抖持久化：合并多次连续修改，延迟单次落盘
 * 每次调用重置定时器，到期后执行一次写入并触发 onScheduleFire
 */
scheduleSave(): void {
  if (this.saveTimer !== -1) {
    clearTimeout(this.saveTimer)
  }
  this.saveTimer = setTimeout(async (): Promise<void> => {
    // 先确保本地落盘完成再触发云同步，避免云端读到旧数据
    await this.persistNow().catch((e: Error): void => {
      Logger.error(this.tag, 'scheduleSave persistNow failed: ' + e.message)
    })
    if (this.onScheduleFire !== null) {
      this.onScheduleFire()
    }
  }, this.delay)
}
/**
 * 立即持久化（页面退出时调用）
 * 清除防抖定时器后立即写入，不触发 onScheduleFire
 */
async flush(): Promise<void> {
  if (this.saveTimer !== -1) {
    clearTimeout(this.saveTimer)
    this.saveTimer = -1
  }
  await this.persistNow()
}
/**
 * 执行实际写入，失败时记录告警日志
 * Preferences API 无事务，此处仅记录不抛出，避免防抖回调中断
 */
private async persistNow(): Promise<void> {
  try {
    await this.persistFn()
  } catch (e) {
    Logger.error(this.tag, 'persistNow failed: ' + (e as Error).message)
  }
}
// 清理定时器，防止页面销毁后定时器仍然触发
```

```
  destroy(): void {
    if (this.saveTimer !== -1) {
      clearTimeout(this.saveTimer)
      this.saveTimer = -1
    }
    Logger.info(this.tag, 'destroy: saveTimer cleared')
  }
}
// 优先级与艾森豪威尔象限的标签/颜色工具函数，统一替代页面间重复实现
import { TaskPriority } from '../../model/BaseTypes'
import { EisenhowerQuadrant } from '../../model/HomeModels'
/** 优先级中文标签（返回 Resource 以支持多语言） */
export function priorityLabelUtil(priority: TaskPriority): Resource {
  if (priority === TaskPriority.HIGH) return $r('app.string.priority_high')
  if (priority === TaskPriority.MEDIUM) return $r('app.string.priority_medium')
  return $r('app.string.priority_low')
}
/** 优先级标签背景色（用于卡片式标签） */
export function priorityBgColorUtil(priority: TaskPriority): Resource {
  if (priority === TaskPriority.HIGH) return $r('app.color.priority_high_bg')
  if (priority === TaskPriority.MEDIUM) return $r('app.color.priority_medium_bg')
  return $r('app.color.priority_low_bg')
}
/** 优先级标签文字色（用于卡片式标签） */
export function priorityTextColorUtil(priority: TaskPriority): Resource {
  if (priority === TaskPriority.HIGH) return $r('app.color.priority_high_text')
  if (priority === TaskPriority.MEDIUM) return $r('app.color.priority_medium_text')
  return $r('app.color.priority_low_text')
}
/** 优先级状态指示色（用于色条/状态标记） */
export function priorityStatusColorUtil(priority: TaskPriority): Resource {
  if (priority === TaskPriority.HIGH) return $r('app.color.error_color')
  if (priority === TaskPriority.MEDIUM) return $r('app.color.warning_color')
  return $r('app.color.text_tertiary')
}
// ==================== 艾森豪威尔四象限工具函数 ====================
// 用于详情页等场景，根据 priority + deadline 动态计算象限并展示标签
// 分类规则须与 HomeViewModel.eisenhowerMatrix 保持一致
/** 紧急度阈值：24 小时（毫秒） */
const URGENT_THRESHOLD_MS: number = 24 * 60 * 60 * 1000
/**
 * 根据 priority（重要度）+ deadline（紧急度）计算艾森豪威尔象限
 * 重要度：HIGH 为重要；紧急度：deadline > 0 且距今不足 24h 为紧急
 * deadline 为 0 视为未设置归入不紧急，已过截止时间也视为紧急
 */
export function classifyEisenhowerQuadrant(priority: TaskPriority, deadline: number): EisenhowerQuadrant {
  const isImportant: boolean = priority === TaskPriority.HIGH
  const hasDeadline: boolean = typeof deadline === 'number' && deadline > 0
  const isUrgent: boolean = hasDeadline && (deadline - Date.now()) < URGENT_THRESHOLD_MS
  if (isImportant && isUrgent) {
```

```
    return EisenhowerQuadrant.Q1_IMPORTANT_URGENT
  } else if (isImportant && !isUrgent) {
    return EisenhowerQuadrant.Q2_IMPORTANT_NOT_URGENT
  } else if (!isImportant && isUrgent) {
    return EisenhowerQuadrant.Q3_NOT_IMPORTANT_URGENT
  }
  return EisenhowerQuadrant.Q4_NOT_IMPORTANT_NOT_URGENT
}
/** 象限中文标签（如"重要且紧急"、"不紧急不重要"等） */
export function quadrantLabelUtil(quadrant: EisenhowerQuadrant): Resource {
  if (quadrant === EisenhowerQuadrant.Q1_IMPORTANT_URGENT) return $r('app.string.eisenhower_q1_title')
  if (quadrant === EisenhowerQuadrant.Q2_IMPORTANT_NOT_URGENT) return $r('app.string.eisenhower_q2_title')
  if (quadrant === EisenhowerQuadrant.Q3_NOT_IMPORTANT_URGENT) return $r('app.string.eisenhower_q3_title')
  return $r('app.string.eisenhower_q4_title')
}
/** 象限标签文字色（Q1红/Q2橙/Q3蓝/Q4灰） */
export function quadrantTextColorUtil(quadrant: EisenhowerQuadrant): Resource {
  if (quadrant === EisenhowerQuadrant.Q1_IMPORTANT_URGENT) return $r('app.color.error_color')
  if (quadrant === EisenhowerQuadrant.Q2_IMPORTANT_NOT_URGENT) return $r('app.color.warning_color')
  if (quadrant === EisenhowerQuadrant.Q3_NOT_IMPORTANT_URGENT) return $r('app.color.info_color')
  return $r('app.color.text_tertiary')
}
/** 象限标签背景色（淡色底：Q1淡红/Q2淡橙/Q3淡蓝/Q4淡灰） */
export function quadrantBgColorUtil(quadrant: EisenhowerQuadrant): Resource {
  if (quadrant === EisenhowerQuadrant.Q1_IMPORTANT_URGENT) return $r('app.color.priority_high_bg')
  if (quadrant === EisenhowerQuadrant.Q2_IMPORTANT_NOT_URGENT) return $r('app.color.priority_medium_bg')
  if (quadrant === EisenhowerQuadrant.Q3_NOT_IMPORTANT_URGENT) return $r('app.color.primary_color_bg')
  return $r('app.color.bg_tag')
}
/**
 * @file 安全区域辅助工具（响应式单例）
 * @description
 *   基于 @ObservedV2 + @Trace 的安全区状态单例，由 EntryAbility 在
 *   initWindowInsets 中初始化并在 avoidAreaChange 回调里 refresh()。
 *   所有 @ComponentV2 组件在 build 中读取本单例即可自动响应
 *   旋转/折叠/分屏后的避让区变化，无需手动订阅。
 *
 * @module common/utils
 */
/**
 * 安全区响应式单例
 *
 * - statusBar: 状态栏高度（vp）
 * - bottom: 底部安全区高度（vp）
 */
@ObservedV2
class SafeAreaHelper {
  /** 状态栏高度（vp） */
  @Trace statusBar: number = 0
  /** 底部安全区高度（vp） */
```

```
  @Trace bottom: number = 0
  /**
   * 获取状态栏高度
   * @returns 状态栏高度（vp），未初始化时返回 0
   */
  getStatusBar(): number {
    return this.statusBar
  }
  /**
   * 获取底部安全区高度
   * @returns 底部安全区高度（vp），未初始化时返回 0
   */
  getBottom(): number {
    return this.bottom
  }
  /**
   * 获取底部安全区高度 + 额外偏移量
   * @param extra 额外偏移量（如导航栏高度 66）
   * @returns 底部安全区高度 + extra
   */
  getBottomWith(extra: number): number {
    return extra + this.bottom
  }
  /**
   * 刷新安全区数值（EntryAbility 初始化与 avoidAreaChange 回调调用）
   * @param status 状态栏高度（vp）
   * @param bottom 底部安全区高度（vp）
   */
  refresh(status: number, bottom: number): void {
    this.statusBar = status
    this.bottom = bottom
  }
}
/** 全局单例 */
export const safeAreaHelper: SafeAreaHelper = new SafeAreaHelper()
// 空值/NaN 安全兜底工具函数，统一替代页面间重复实现
import { resourceManager } from '@kit.LocalizationKit'
/** 空值/空字符串安全兜底，非有效字符串时返回 fallback */
export function safeStringUtil(value: string | null | undefined, fallback: string): string {
  if (typeof value === 'string' && value.length > 0) return value
  return fallback
}
/** NaN/非数字安全兜底，有效数字返回非负整数取整值，否则返回 fallback */
export function safeNumberUtil(value: number | null | undefined, fallback: number = 0): number {
  if (typeof value === 'number' && !isNaN(value)) return Math.max(0, Math.floor(value))
  return fallback
}
/** 两位补零 */
function pad2Util(v: number): string {
  return v < 10 ? '0' + v.toString() : v.toString()
```

```
}
/**
 * 截止时间毫秒时间戳 → "YYYY-MM-DD" 文本
 * 无效/0/NaN 返回 "--"（表示未设置截止时间）
 */
export function formatDeadlineUtil(timestamp: number | null | undefined): string {
  if (typeof timestamp !== 'number' || isNaN(timestamp) || timestamp <= 0) return '--'
  const d: Date = new Date(timestamp)
  return d.getFullYear().toString() + '-' + pad2Util(d.getMonth() + 1) + '-' + pad2Util(d.getDate())
}
/**
 * 分钟数 → 智能耗时文案：≥1 天按"X 天"、≥1 小时按"X 小时"、否则"X 分钟"
 * （资源单位随系统语言；resourceManager 不可用时回退英文，与历史 ' min' 行为兼容）
 * @param minutes - 预估耗时（分钟），无效/0 返回空串
 * @param rm - 资源管理器，可空
 */
export function formatEstimatedDuration(minutes: number | null | undefined,
  rm: resourceManager.ResourceManager | null | undefined): string {
  const safeMinutes: number = (typeof minutes === 'number' && !isNaN(minutes) && minutes > 0)
    ? Math.floor(minutes) : 0
  if (safeMinutes === 0) return ''
  let unitText: string
  let value: number
  if (safeMinutes >= 1440) {
    value = Math.round(safeMinutes / 1440)
    unitText = rm ? rm.getStringByNameSync('duration_unit_day') : ' days'
  } else if (safeMinutes >= 60) {
    value = Math.round(safeMinutes / 60)
    unitText = rm ? rm.getStringByNameSync('duration_unit_hour') : ' hrs'
  } else {
    value = safeMinutes
    unitText = rm ? rm.getStringByNameSync('duration_unit_min') : ' min'
  }
  return value.toString() + unitText
}
/**
 * 统一存储管理器 - 封装 ArkData preferences，提供类型安全的 JSON 序列化/反序列化
 * 关键设计：LRU 缓存 Preferences 实例、写锁串行化避免并发冲突、数据损坏时静默回退 fallback
 */
import { preferences } from '@kit.ArkData'
import { common } from '@kit.AbilityKit'
import { BusinessError } from './BusinessError'
import { Logger } from './Logger'
import { TaskPriority } from '../../model/Models'
import { parseTaskPriority } from '../../model/BaseTypes'
import { AiGeneratedTask, AiGeneratedTaskJSON } from '../../model/Models'
import { CommonConstants } from '../constants/CommonConstants'
// 顶层类型声明
/**
 * 存储操作结果（统一返回类型，避免 throw）
```

```
 */
export interface StorageResult {
  success: boolean
  errorMsg: string
}
/**
 * 缓存条目
 */
interface CacheEntry {
  prefs: preferences.Preferences
  lastAccessTime: number
}
interface StringWrapper {
  v: string
}
interface BooleanWrapper {
  v: boolean
}
// 常量定义
const TAG: string = 'StorageManager'
const STORE_NAME: string = 'app_data_store'
/** 分域 store：大 JSON 与高频写入的数据拆分到独立 preferences 文件，
 *  避免任何一次小写入都触发整包大值重写（历史上所有键共用单一 store 文件） */
const STORE_NAME_GOALS: string = 'goals_store'
const STORE_NAME_TODOS: string = 'todos_store'
const STORE_NAME_NOTIFY: string = 'notify_store'
const STORE_NAME_POMO: string = 'pomo_store'
/**
 * 按存储键前缀路由到目标 store（StorageManager 内部使用，调用方无感）
 * - 目标/通知/番茄钟：写入频繁，与目标大 JSON 物理隔离
 * - 目标列表：全项目最大 JSON，单独成库
 */
function resolveStoreName(key: string): string {
  if (key.indexOf('storage_route_goals') === 0) {
    return STORE_NAME_GOALS
  }
  if (key.indexOf('storage_home_todos') === 0) {
    return STORE_NAME_TODOS
  }
  if (key.indexOf('storage_app_notifications') === 0) {
    return STORE_NAME_NOTIFY
  }
  if (key.indexOf('storage_pomodoro') === 0 || key.indexOf('storage_pomo_') === 0) {
    return STORE_NAME_POMO
  }
  return STORE_NAME
}
/** 缓存过期时间（毫秒） */
const CACHE_TTL_MS: number = 30000
/** 缓存最大条目数 */
```

```
const MAX_CACHE_SIZE: number = 16
// StorageManager 主体
/**
 * 统一存储管理器（静态工具类）
 */
export class StorageManager {
  /**
   * Preferences 实例缓存（key = context 标识）
   * 用 Map 维护，按 context 哈希 + 最后访问时间排序
   */
  private static cache: Map<string, CacheEntry> = new Map()
  /**
   * 写锁队列：保证同一 key 的写操作串行化
   * key = 存储键名
   */
  private static writeLocks: Map<string, Promise<void>> = new Map()
  /** 私有构造：禁止实例化 */
  private constructor() {}
  /**
   * 获取（或复用）Preferences 实例
   * @param context - 应用上下文（getContext(this) 获取）
   * @returns Promise<preferences.Preferences>
   */
  private static async getPreferences(context: Context,
    storeName: string = STORE_NAME): Promise<preferences.Preferences> {
    const cacheKey: string = StorageManager.buildCacheKey(context) + '|' + storeName
    const now: number = Date.now()
    // 1. 检查缓存（命中且未过期）
    const cached: CacheEntry | undefined = StorageManager.cache.get(cacheKey)
    if (cached !== undefined && (now - cached.lastAccessTime) < CACHE_TTL_MS) {
      // 更新访问时间
      const refreshed: CacheEntry = {
        prefs: cached.prefs,
        lastAccessTime: now
      }
      StorageManager.cache.set(cacheKey, refreshed)
      return cached.prefs
    }
    // 2. 缓存未命中或已过期，重新获取
    try {
      const prefs: preferences.Preferences =
        await preferences.getPreferences(context, storeName)
      // 3. 写入缓存
      StorageManager.cache.set(cacheKey, {
        prefs: prefs,
        lastAccessTime: now
      })
      // 4. 缓存超过上限，清理最旧条目
      if (StorageManager.cache.size > MAX_CACHE_SIZE) {
        StorageManager.evictOldest()
```

```
    }
    Logger.info(TAG, 'getPreferences success, cacheKey=' + cacheKey)
    return prefs
  } catch (e) {
    const err: BusinessError = e as BusinessError
    Logger.error(TAG, 'getPreferences failed, code=' + err.code.toString() +
      ', msg=' + err.message)
    throw new Error('StorageManager.getPreferences failed: ' + err.message)
  }
}
/**
 * 构建缓存键（避免长生命周期 context 引用）
 * 优先使用 context 中可序列化的标识
 */
private static buildCacheKey(context: Context): string {
  // UIAbilityContext 有 cacheDir 等稳定标识
  if (context !== null && context !== undefined) {
    const ctx: Context = context
    return typeof ctx.cacheDir === 'string' ?
      ctx.cacheDir : 'ctx_' + StorageManager.hashCode(JSON.stringify({
        stage: (ctx as common.UIAbilityContext).stageMode,
        process: (ctx as common.UIAbilityContext).processName
      })).toString()
  }
  return 'ctx_null'
}
/**
 * 字符串哈希（简单实现，避免引用完整 hash 库）
 */
private static hashCode(str: string): number {
  let hash: number = 0
  for (let i: number = 0; i < str.length; i++) {
    hash = ((hash << 5) - hash) + str.charCodeAt(i)
    hash = hash & hash // 转为 32 位整数
  }
  return Math.abs(hash)
}
/**
 * 清理最旧的缓存条目
 */
private static evictOldest(): void {
  let oldestKey: string = ''
  let oldestTime: number = Number.MAX_SAFE_INTEGER
  StorageManager.cache.forEach((entry: CacheEntry, key: string): void => {
    if (entry.lastAccessTime < oldestTime) {
      oldestTime = entry.lastAccessTime
      oldestKey = key
    }
  })
  if (oldestKey.length > 0) {
```

```
      StorageManager.cache.delete(oldestKey)
    }
  }
  /**
   * 核心方法：保存数据到 Preferences
   *
   * @param context - 应用上下文（getContext(this)）
   * @param key - 存储键名
   * @param value - 要存储的对象（必须可 JSON 序列化）
   * @returns Promise<void> 存储完成的 Promise
   *
   * @example
   * ```typescript
   * await StorageManager.saveData(
   *   getContext(this),
   *   CommonConstants.STORAGE_KEY_HOME_TODOS,
   *   this.taskList
   * )
   * ```
   */
  static async saveData(context: Context, key: string, value: object): Promise<void> {
    if (context === null || context === undefined) {
      Logger.error(TAG, 'saveData: context is null')
      return
    }
    if (typeof key !== 'string' || key.length === 0) {
      Logger.error(TAG, 'saveData: key is empty')
      return
    }
    if (value === null || value === undefined) {
      Logger.error(TAG, 'saveData: value is null, key=' + key)
      return
    }
    // 写锁串行化：同一 key 的写操作排队执行，锁的获取与注册在同一微任务内完成避免 TOCTOU 竞态
    const existingLock: Promise<void> | undefined = StorageManager.writeLocks.get(key)
    const writePromise: Promise<void> = (async (): Promise<void> => {
      // 先等待之前的写操作完成（在 writePromise 内部 await，不阻塞锁注册）
      if (existingLock !== undefined) {
        await existingLock
      }
      try {
        const prefs: preferences.Preferences =
          await StorageManager.getPreferences(context, resolveStoreName(key))
        const jsonStr: string = JSON.stringify(value)
        await prefs.put(key, jsonStr)
        await prefs.flush()
        Logger.info(TAG, 'saveData success, key=' + key +
          ', size=' + jsonStr.length.toString())
      } catch (e) {
        const err: BusinessError = e as BusinessError
```

```
      Logger.error(TAG, 'saveData failed, key=' + key +
        ', code=' + err.code.toString() + ', msg=' + err.message)
    }
  })()
  // 立即注册锁（在任何 await 之前，确保并发调用能看到新锁）
  StorageManager.writeLocks.set(key, writePromise)
  try {
    await writePromise
  } finally {
    // 释放写锁
    if (StorageManager.writeLocks.get(key) === writePromise) {
      StorageManager.writeLocks.delete(key)
    }
  }
}
/**
 * 核心方法：从 Preferences 加载数据
 *
 * @typeParam T - 期望返回的数据类型
 * @param context - 应用上下文
 * @param key - 存储键名
 * @param fallback - 默认值（加载失败或数据不存在时返回）
 * @returns Promise<T> 包含加载数据的 Promise
 *
 * @example
 * ```typescript
 * const todos: Array<HomeTodoItem> = await StorageManager.loadData<Array<HomeTodoItem>>(
 *   getContext(this),
 *   CommonConstants.STORAGE_KEY_HOME_TODOS,
 *   []
 * )
 * ```
 */
static async loadData<T>(context: Context, key: string, fallback: T): Promise<T> {
  if (context === null || context === undefined) {
    Logger.error(TAG, 'loadData: context is null')
    return fallback
  }
  if (typeof key !== 'string' || key.length === 0) {
    Logger.error(TAG, 'loadData: key is empty')
    return fallback
  }
  try {
    const prefs: preferences.Preferences =
      await StorageManager.getPreferences(context, resolveStoreName(key))
    const hasKey: boolean = await prefs.has(key)
    if (!hasKey) {
      Logger.info(TAG, 'loadData: key not found, key=' + key + ', use fallback')
      return fallback
    }
```

```
      const jsonStr: string = await prefs.get(key, '') as string
      if (typeof jsonStr !== 'string' || jsonStr.length === 0) {
        Logger.warn(TAG, 'loadData: empty value, key=' + key)
        return fallback
      }
      // JSON.parse 单独 try-catch：数据损坏时返回 fallback，不抛异常保证 UI 不崩
      try {
        const parsed: T = JSON.parse(jsonStr) as T
        Logger.info(TAG, 'loadData success, key=' + key)
        return parsed
      } catch (parseError) {
        Logger.error(TAG, 'loadData: JSON.parse failed (data corruption), key=' + key +
          ', rawLen=' + jsonStr.length.toString() +
          ', msg=' + (parseError as Error).message +
          ' (已返回 fallback 默认值)')
        return fallback
      }
    } catch (e) {
      const err: BusinessError = e as BusinessError
      Logger.error(TAG, 'loadData failed, key=' + key +
        ', code=' + err.code.toString() + ', msg=' + err.message)
      return fallback
    }
  }
  /**
   * 同步读取指定键的数据（卡片场景专用）
   * @param context - 应用上下文
   * @param key - 存储键名
   * @param fallback - 默认值（加载失败或数据不存在时返回）
   * @returns T 包含加载数据的对象
   *
   * 说明：卡片 FormExtensionAbility 的 onAddForm 是同步回调，无法使用 async/await，
   * 因此提供同步版本。底层使用 preferences.getPreferencesSync + getSync。
   */
  static loadDataSync<T>(context: Context, key: string, fallback: T): T {
    if (context === null || context === undefined) {
      Logger.error(TAG, 'loadDataSync: context is null')
      return fallback
    }
    if (typeof key !== 'string' || key.length === 0) {
      Logger.error(TAG, 'loadDataSync: key is empty')
      return fallback
    }
    try {
      // API 23 同步获取 Preferences 实例：options 是对象 { name: string }
      // 桌面卡片读取的 todos 键在分域后位于 todos_store，必须同步路由
      const options: preferences.Options = { name: resolveStoreName(key) }
      const prefs: preferences.Preferences = preferences.getPreferencesSync(context, options)
      const hasKey: boolean = prefs.hasSync(key)
      if (!hasKey) {
```

```
      Logger.info(TAG, 'loadDataSync: key not found, key=' + key + ', use fallback')
      return fallback
    }
    const jsonStr: string = prefs.getSync(key, '') as string
    if (typeof jsonStr !== 'string' || jsonStr.length === 0) {
      Logger.warn(TAG, 'loadDataSync: empty value, key=' + key)
      return fallback
    }
    try {
      const parsed: T = JSON.parse(jsonStr) as T
      Logger.info(TAG, 'loadDataSync success, key=' + key)
      return parsed
    } catch (parseError) {
      Logger.error(TAG, 'loadDataSync: JSON.parse failed (data corruption), key=' + key +
        ', msg=' + (parseError as Error).message)
      return fallback
    }
  } catch (e) {
    const err: BusinessError = e as BusinessError
    Logger.error(TAG, 'loadDataSync failed, key=' + key +
      ', code=' + err.code.toString() + ', msg=' + err.message)
    return fallback
  }
}
/**
 * 删除指定键
 * @param context - 应用上下文
 * @param key - 存储键名
 * @returns Promise<StorageResult> 操作结果
 */
static async removeData(context: Context, key: string): Promise<StorageResult> {
  const result: StorageResult = { success: false, errorMsg: '' }
  if (context === null || context === undefined) {
    result.errorMsg = 'context is null'
    Logger.error(TAG, 'removeData: ' + result.errorMsg)
    return result
  }
  if (typeof key !== 'string' || key.length === 0) {
    result.errorMsg = 'key is empty'
    Logger.error(TAG, 'removeData: ' + result.errorMsg)
    return result
  }
  try {
    const prefs: preferences.Preferences =
      await StorageManager.getPreferences(context, resolveStoreName(key))
    await prefs.delete(key)
    await prefs.flush()
    result.success = true
    Logger.info(TAG, 'removeData success, key=' + key)
  } catch (e) {
```

```
    const err: BusinessError = e as BusinessError
    result.errorMsg = err.message
    Logger.error(TAG, 'removeData failed, key=' + key +
      ', code=' + err.code.toString() + ', msg=' + err.message)
  }
  return result
}
/**
 * 检查键是否存在
 * @param context - 应用上下文
 * @param key - 存储键名
 * @returns Promise<boolean> 是否存在
 */
static async hasData(context: Context, key: string): Promise<boolean> {
  if (context === null || context === undefined || typeof key !== 'string' || key.length === 0) {
    return false
  }
  try {
    const prefs: preferences.Preferences =
      await StorageManager.getPreferences(context, resolveStoreName(key))
    return await prefs.has(key)
  } catch (e) {
    const err: BusinessError = e as BusinessError
    Logger.error(TAG, 'hasData failed, key=' + key + ', msg=' + err.message)
    return false
  }
}
/**
 * 清除所有缓存（应用退出前调用，避免 context 泄漏）
 */
static clearCache(): void {
  StorageManager.cache.clear()
  Logger.info(TAG, 'clearCache done')
}
// 兼容层：统一 PreferencesUtil / PreferenceManager 调用入口
/** 应用上下文缓存 */
private static appContext: Context | null = null
/** 分域迁移是否已完成（每进程一次） */
private static storeMigrationDone: boolean = false
/**
 * 兼容层：初始化 Preferences（替代 PreferencesUtil.init + PreferenceManager.init）
 * 统一入口：EntryAbility.onCreate 只需调用 StorageManager.init(context)
 *
 * @param context - UIAbilityContext
 */
static async init(context: common.UIAbilityContext): Promise<void> {
  StorageManager.appContext = context;
  // 分域存储迁移：每进程最多执行一次（幂等）
  if (!StorageManager.storeMigrationDone) {
    await StorageManager.migrateToSplitStores(context)
```

```
      StorageManager.storeMigrationDone = true
    }
    Logger.info(TAG, 'init success');
  }
  /**
   * 一次性迁移：把历史版本存放在默认 store 中的分域键搬到目标 store，并从默认 store 删除
   * 安全性：逐键先 put 后 delete——任一步失败都不会丢数据（最坏情况双份，读路径以新 store 为准）
   */
  private static async migrateToSplitStores(context: Context): Promise<void> {
    try {
      const defaultPrefs: preferences.Preferences =
        await StorageManager.getPreferences(context, STORE_NAME)
      const allObj: Object = await defaultPrefs.getAll()
      const keys: Array<string> = Object.keys(allObj as Record<string, Object>)
      let moved: number = 0
      for (let i: number = 0; i < keys.length; i++) {
        const key: string = keys[i]
        const target: string = resolveStoreName(key)
        if (target === STORE_NAME) {
          continue
        }
        const value: Object = (allObj as Record<string, Object>)[key]
        const targetPrefs: preferences.Preferences =
          await StorageManager.getPreferences(context, target)
        const hasInTarget: boolean = await targetPrefs.has(key)
        if (!hasInTarget) {
          await targetPrefs.put(key, value)
          await targetPrefs.flush()
        }
        await defaultPrefs.delete(key)
        moved++
      }
      if (moved > 0) {
        await defaultPrefs.flush()
      }
      Logger.info(TAG, 'migrateToSplitStores done, moved=' + moved.toString())
    } catch (e) {
      // 迁移失败不阻塞启动：数据仍留在默认 store，下次启动重试
      Logger.error(TAG, 'migrateToSplitStores failed: ' + (e as Error).message)
    }
  }
  static async setPreference(key: string, value: string): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'setPreference: appContext not initialized')
      return
    }
    const wrapper: StringWrapper = { v: value }
    await StorageManager.saveData(StorageManager.appContext, key, wrapper)
  }
  static async getPreference(key: string, defaultValue: string): Promise<string> {
```

```
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'getPreference: appContext not initialized')
      return defaultValue
    }
    const fallback: StringWrapper = { v: defaultValue }
    const wrapper: StringWrapper = await StorageManager.loadData<StringWrapper>(
      StorageManager.appContext, key, fallback)
    return wrapper.v
  }
  static async setPreferenceBool(key: string, value: boolean): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'setPreferenceBool: appContext not initialized')
      return
    }
    const wrapper: BooleanWrapper = { v: value }
    await StorageManager.saveData(StorageManager.appContext, key, wrapper)
  }
  static async getPreferenceBool(key: string, defaultValue: boolean): Promise<boolean> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'getPreferenceBool: appContext not initialized')
      return defaultValue
    }
    const fallback: BooleanWrapper = { v: defaultValue }
    const wrapper: BooleanWrapper = await StorageManager.loadData<BooleanWrapper>(
      StorageManager.appContext, key, fallback)
    return wrapper.v
  }
  static async setPreferenceObject(key: string, obj: Object): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'setPreferenceObject: appContext not initialized')
      return
    }
    await StorageManager.saveData(StorageManager.appContext, key, obj)
  }
  static async getPreferenceObject<T>(key: string): Promise<T | null> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'getPreferenceObject: appContext not initialized')
      return null
    }
    return await StorageManager.loadData<T | null>(StorageManager.appContext, key, null)
  }
  static async hasPreference(key: string): Promise<boolean> {
    if (StorageManager.appContext === null) {
      return false
    }
    return await StorageManager.hasData(StorageManager.appContext, key)
  }
  static async deletePreference(key: string): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'deletePreference: appContext not initialized')
```

```
      return
    }
    await StorageManager.removeData(StorageManager.appContext, key)
  }
  /**
   * 兼容层：保存 AI 拆解任务（替代 PreferenceManager.saveAITasks）
   * 内部仍委托 StorageManager.saveData，统一入口
   *
   * @param tasks - AI 拆解任务数组
   */
  static async saveAITasks(tasks: Array<AiGeneratedTask>): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'saveAITasks: context not initialized')
      return
    }
    try {
      const snapshot: Array<object> = []
      for (let i: number = 0; i < tasks.length; i++) {
        snapshot.push(tasks[i].toJSON() as object)
      }
      await StorageManager.saveData(
        StorageManager.appContext,
        CommonConstants.STORAGE_KEY_AI_TASKS,
        snapshot as object
      )
      Logger.info(TAG, 'saveAITasks success, count=' + tasks.length.toString())
    } catch (e) {
      Logger.error(TAG, 'saveAITasks failed: ' + (e as Error).message)
    }
  }
  /**
   * 兼容层：加载 AI 拆解任务（替代 PreferenceManager.loadAITasks）
   * 内部仍委托 StorageManager.loadData，统一入口
   *
   * @returns AI 拆解任务数组
   */
  static async loadAITasks(): Promise<Array<AiGeneratedTask>> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'loadAITasks: context not initialized')
      return []
    }
    try {
      const raw: Array<AiGeneratedTaskJSON> = await StorageManager.loadData<Array<AiGeneratedTaskJSON>>(
        StorageManager.appContext,
        CommonConstants.STORAGE_KEY_AI_TASKS,
        []
      )
      if (!Array.isArray(raw) || raw.length === 0) {
        return []
      }
```

```
      const tasks: Array<AiGeneratedTask> = []
      for (let i: number = 0; i < raw.length; i++) {
        const task: AiGeneratedTask | null = AiGeneratedTask.fromJSON(raw[i])
        if (task !== null) {
          tasks.push(task)
        } else {
          Logger.warn(TAG, 'loadAITasks: skip invalid record at index=' + i.toString())
        }
      }
      Logger.info(TAG, 'loadAITasks success, count=' + tasks.length.toString())
      return tasks
    } catch (e) {
      Logger.error(TAG, 'loadAITasks failed: ' + (e as Error).message)
      return []
    }
  }
  /**
   * 兼容层：清除 AI 拆解任务（替代 PreferenceManager.clearAITasks）
   * 内部仍委托 StorageManager.removeData，统一入口
   */
  static async clearAITasks(): Promise<void> {
    if (StorageManager.appContext === null) {
      Logger.error(TAG, 'clearAITasks: context not initialized')
      return
    }
    try {
      await StorageManager.removeData(
        StorageManager.appContext,
        CommonConstants.STORAGE_KEY_AI_TASKS
      )
      Logger.info(TAG, 'clearAITasks success')
    } catch (e) {
      Logger.error(TAG, 'clearAITasks failed: ' + (e as Error).message)
    }
  }
  /**
   * 兼容层：保存草稿（替代 AIPage 中 AppStorage 草稿操作）
   * 底层已从 AppStorage 迁移至 PreferencesUtil 持久化
   *
   * @param draftKey - 草稿存储键名
   * @param text - 草稿文本
   */
  static saveDraft(draftKey: string, text: string): void {
    if (StorageManager.appContext === null) {
      Logger.warn(TAG, 'saveDraft: appContext not initialized')
      return
    }
    try {
      if (text.length > 0) {
        const wrapper: StringWrapper = { v: text }
```

```
      StorageManager.saveData(StorageManager.appContext, draftKey, wrapper)
    } else {
      StorageManager.removeData(StorageManager.appContext, draftKey)
    }
  } catch (e) {
    Logger.warn(TAG, 'saveDraft failed: ' + (e as Error).message)
  }
}
/**
 * 兼容层：加载草稿（替代 AIPage 中 AppStorage 草稿操作）
 * 底层已从 AppStorage 迁移至 PreferencesUtil 持久化
 *
 * @param draftKey - 草稿存储键名
 * @returns 草稿文本
 */
static async loadDraft(draftKey: string): Promise<string> {
  if (StorageManager.appContext === null) {
    Logger.warn(TAG, 'loadDraft: appContext not initialized')
    return ''
  }
  try {
    const fallback: StringWrapper = { v: '' }
    const wrapper: StringWrapper = await StorageManager.loadData<StringWrapper>(
      StorageManager.appContext, draftKey, fallback)
    return wrapper.v
  } catch (e) {
    Logger.warn(TAG, 'loadDraft failed: ' + (e as Error).message)
    return ''
  }
}
/**
 * 兼容层：清除草稿（替代 AIPage 中 AppStorage.delete 草稿操作）
 * 底层已从 AppStorage 迁移至 PreferencesUtil 持久化
 *
 * @param draftKey - 草稿存储键名
 */
static clearDraft(draftKey: string): void {
  if (StorageManager.appContext === null) {
    Logger.warn(TAG, 'clearDraft: appContext not initialized')
    return
  }
  StorageManager.removeData(StorageManager.appContext, draftKey)
}
/**
 * 兼容层：检查是否已初始化（替代 PreferenceManager.isInitialized）
 *
 * @returns 是否已初始化
 */
static isInitialized(): boolean {
  return StorageManager.appContext !== null
```

```
  }
  /**
   * 工具方法：解析 TaskPriority 枚举值（兼容 number/string）
   * JSON 序列化后 enum 会变成字符串，必须做兼容
   * @param raw - 序列化后的原始值
   * @returns TaskPriority 枚举值
   *
   * 注意：此方法保留为兼容代理，实际逻辑已迁移至 Model 层 parseTaskPriority
   */
  static parseTaskPriority(raw: number | string | null | undefined): TaskPriority {
    return parseTaskPriority(raw)
  }
}
/**
 * @file WantAgent 工厂（通知点击直达路由）
 * @description
 *   从 ProfileViewModel 下沉至 common 层：
 *   PomodoroService / PomodoroNotificationService / DeadlineReminderService 等服务层
 *   需要构造通知 WantAgent，此前反向 import viewmodel 造成层级倒挂。
 *   下沉后服务层只依赖 common，viewmodel 亦可复用。
 *
 * - createNotificationWantAgent: 点击通知仅拉起应用到首页
 * - createGoalDetailWantAgent: 点击通知直达目标详情页（携带 goalId）
 * - createTaskDetailWantAgent: 点击通知直达任务详情页（携带 taskId）
 */
import { wantAgent, WantAgent, common } from '@kit.AbilityKit'
/**
 * 构造目标详情页路由参数（Want.parameters 类型为 Record<string, Object>）
 */
function buildGoalRouteParams(goalId: string): Record<string, Object> {
  const params: Record<string, Object> = {}
  params['routeName'] = 'GoalDetailPage'
  params['goalId'] = goalId
  return params
}
/**
 * 构造任务详情页路由参数（Want.parameters 类型为 Record<string, Object>）
 *
 * @param taskId - 任务 ID
 * @param openImmersive - 是否需要落地即呼出全屏番茄钟（番茄钟通知专用）
 */
function buildTaskRouteParams(taskId: string, openImmersive: boolean): Record<string, Object> {
  const params: Record<string, Object> = {}
  params['routeName'] = 'TaskDetailPage'
  params['taskId'] = taskId
  // 显式标记"从番茄钟通知进来"：EntryAbility 收到后直接 showImmersive()，
  // 不依赖进程内 isImmersiveVisible 标志（进程被回收后该标志为 false，点通知只会落到任务详情）
  if (openImmersive) {
    params['openImmersive'] = '1'
  }
```

```
  return params
}
export class WantAgentFactory {
  static async createNotificationWantAgent(context: Context): Promise<WantAgent> {
    const uiaContext: common.UIAbilityContext = context as common.UIAbilityContext
    const wantAgentInfo: wantAgent.WantAgentInfo = {
      wants: [
        {
          deviceId: '',
          bundleName: uiaContext.abilityInfo.bundleName,
          abilityName: uiaContext.abilityInfo.name
        }
      ],
      actionType: wantAgent.OperationType.START_ABILITY,
      requestCode: 0,
      actionFlags: [wantAgent.WantAgentFlags.UPDATE_PRESENT_FLAG]
    }
    return await wantAgent.getWantAgent(wantAgentInfo)
  }
  /**
   * 创建点击通知后直达目标详情页的 WantAgent
   * 携带 parameters: { routeName: 'GoalDetailPage', goalId: xxx }
   * EntryAbility.onNewWant / onCreate 解析后调用 navigateToWithId 跳转
   *
   * @param context - 应用上下文
   * @param goalId - 目标 ID
   * @returns WantAgent 实例
   */
  static async createGoalDetailWantAgent(context: Context, goalId: string): Promise<WantAgent> {
    const uiaContext: common.UIAbilityContext = context as common.UIAbilityContext
    const routeParams: Record<string, Object> = buildGoalRouteParams(goalId)
    const wantAgentInfo: wantAgent.WantAgentInfo = {
      wants: [
        {
          deviceId: '',
          bundleName: uiaContext.abilityInfo.bundleName,
          abilityName: uiaContext.abilityInfo.name,
          parameters: routeParams
        }
      ],
      actionType: wantAgent.OperationType.START_ABILITY,
      requestCode: 0,
      actionFlags: [wantAgent.WantAgentFlags.UPDATE_PRESENT_FLAG]
    }
    return await wantAgent.getWantAgent(wantAgentInfo)
  }
  /**
   * 创建点击通知后直达任务详情页的 WantAgent
   * 携带 parameters: { routeName: 'TaskDetailPage', taskId: xxx[, openImmersive: '1'] }
   * EntryAbility.onNewWant / onCreate 解析后调用 navigateToWithIdOnce 跳转
```

```
   *
   * @param context - 应用上下文
   * @param taskId - 任务 ID
   * @param openImmersive - true 时落地即呼出全屏番茄钟（仅番茄钟倒计时通知传 true）
   * @returns WantAgent 实例
   */
  static async createTaskDetailWantAgent(context: Context, taskId: string,
    openImmersive: boolean = false): Promise<WantAgent> {
    const uiaContext: common.UIAbilityContext = context as common.UIAbilityContext
    const routeParams: Record<string, Object> = buildTaskRouteParams(taskId, openImmersive)
    const wantAgentInfo: wantAgent.WantAgentInfo = {
      wants: [
        {
          deviceId: '',
          bundleName: uiaContext.abilityInfo.bundleName,
          abilityName: uiaContext.abilityInfo.name,
          parameters: routeParams
        }
      ],
      actionType: wantAgent.OperationType.START_ABILITY,
      requestCode: 0,
      actionFlags: [wantAgent.WantAgentFlags.UPDATE_PRESENT_FLAG]
    }
    return await wantAgent.getWantAgent(wantAgentInfo)
  }
}
import { FormExtensionAbility, formBindingData, formProvider } from '@kit.FormKit'
import { Want } from '@kit.AbilityKit'
import { Configuration } from '@kit.AbilityKit'
import { fileIo as fs } from '@kit.CoreFileKit'
import { util } from '@kit.ArkTS'
import { preferences } from '@kit.ArkData'
import { Logger } from '../common/utils/Logger'
/** 卡片完整数据结构 */
interface TodoCardBindingData {
  totalCount: number
  completedCount: number
  task1Title: string
  task2Title: string
  hasTasks: boolean
  completionRate: number
  remainingCount: number
  showTask1: boolean
  showTask2: boolean
  showDivider: boolean
  showMore: boolean
  showProgress: boolean
  showHeaderStats: boolean
}
const TAG: string = 'EntryFormAbility'
```

```
const PREF_NAME: string = 'momentum_card_store'
export default class EntryFormAbility extends FormExtensionAbility {
  onAddForm(want: Want): formBindingData.FormBindingData {
    const params: Record<string, Object> = want.parameters as Record<string, Object>
    const formId: string = (params['ohos.extra.param.key.form_identity'] as string)
      || (params['formId'] as string)
      || ''
    Logger.info(TAG, 'onAddForm: formId=' + formId)
    if (formId.length > 0) {
      this.persistFormId(formId).catch((e: Error): void => {
        Logger.error(TAG, 'persistFormId failed: ' + e.message)
      })
    }
    const bindingData: TodoCardBindingData = this.loadSnapshot()
    const fbData: formBindingData.FormBindingData =
      formBindingData.createFormBindingData(bindingData)
    if (formId.length > 0) {
      formProvider.updateForm(formId, fbData)
        .then((): void => { Logger.info(TAG, 'onAddForm push success') })
        .catch((e: Error): void => { Logger.error(TAG, 'onAddForm push failed: ' + e.message) })
    }
    return fbData
  }
  onUpdateForm(formId: string): formBindingData.FormBindingData {
    Logger.info(TAG, 'onUpdateForm: formId=' + formId)
    const bindingData: TodoCardBindingData = this.loadSnapshot()
    const fbData: formBindingData.FormBindingData =
      formBindingData.createFormBindingData(bindingData)
    formProvider.updateForm(formId, fbData)
      .then((): void => { Logger.info(TAG, 'onUpdateForm push success') })
      .catch((e: Error): void => { Logger.error(TAG, 'onUpdateForm push failed: ' + e.message) })
    return fbData
  }
  private loadSnapshot(): TodoCardBindingData {
    const emptyData: TodoCardBindingData = {
      totalCount: 0, completedCount: 0, task1Title: '', task2Title: '',
      hasTasks: false, completionRate: 0, remainingCount: 0,
      showTask1: false, showTask2: false, showDivider: false,
      showMore: false, showProgress: false, showHeaderStats: false
    }
    try {
      const appCtx = this.context.getApplicationContext()
      const path: string = appCtx.filesDir + '/card_snapshot.json'
      if (!fs.accessSync(path)) {
        return emptyData
      }
      const stat = fs.statSync(path)
      const file = fs.openSync(path, fs.OpenMode.READ_ONLY)
      const buffer = new ArrayBuffer(stat.size)
      fs.readSync(file.fd, buffer)
```

```
      fs.closeSync(file)
      const textDecoder = new util.TextDecoder('utf-8')
      const snapshotJson: string = textDecoder.decodeToString(new Uint8Array(buffer))
      if (snapshotJson.length === 0) {
        return emptyData
      }
      // HomeViewModel 已写入完整快照数据，直接解析返回
      const snapshot: TodoCardBindingData = JSON.parse(snapshotJson) as TodoCardBindingData
      if (snapshot === null || typeof snapshot.totalCount !== 'number') {
        return emptyData
      }
      Logger.info(TAG, 'loadSnapshot: total=' + snapshot.totalCount.toString() +
        ', rate=' + snapshot.completionRate.toString())
      return snapshot
    } catch (e) {
      Logger.error(TAG, 'loadSnapshot failed: ' + (e as Error).message)
      return emptyData
    }
  }
  private async persistFormId(formId: string): Promise<void> {
    const prefs = await preferences.getPreferences(this.context, PREF_NAME)
    await prefs.put('form_id', formId)
    await prefs.flush()
    Logger.info(TAG, 'persistFormId: saved formId=' + formId)
  }
  onFormEvent(formId: string, message: string): void {
    Logger.info(TAG, 'onFormEvent: formId=' + formId)
  }
  onRemoveForm(formId: string): void {
    Logger.info(TAG, 'onRemoveForm: formId=' + formId)
    this.clearPersistedFormId().catch((e: Error): void => {
      Logger.error(TAG, 'clearPersistedFormId failed: ' + e.message)
    })
  }
  onConfigurationUpdate(config: Configuration): void {
    Logger.info(TAG, 'onConfigurationUpdate')
  }
  private async clearPersistedFormId(): Promise<void> {
    const prefs = await preferences.getPreferences(this.context, PREF_NAME)
    await prefs.delete('form_id')
    await prefs.flush()
  }
}
```